

Development of Flow at the Entrance of a Cylindrical Pipe

Numerical Simulation and Modelling of Incompressible Flow

LV-Nr.: 321.053 EX
Winter Semester 2025/26

Felix Scope
`felix_scope@yahoo.de`

September 18, 2026

Contents

Assignment	3
Statement on the Use of AI Tools	6
1 Problem Description	7
1.1 Geometry and Setup	7
1.2 Physical Parameters	7
2 Mathematical Formulation	8
2.1 Governing Equations in Cylindrical Coordinates	8
2.1.1 Continuity Equation	8
2.1.2 Momentum Equations	8
2.1.3 Energy Equation	9
2.2 Boundary Conditions	9
2.3 Non-Dimensionalization	10
2.3.1 Reference Scales	10
2.3.2 Non-Dimensional Variables	10
2.3.3 Non-Dimensional Governing Equations	10
2.3.4 Non-Dimensional Boundary Conditions	10
3 Numerical Solution	11
3.1 Computational Domain and Grid	11
3.2 Spatial Discretization: Finite Volume Method	11
3.2.1 Continuity (at scalar CV)	11
3.2.2 Convective Scheme: 2nd-Order Upwind (Linear Upwind Differencing)	12
3.2.3 Temperature equation and cell-by-cell development	13
3.2.4 Axial Momentum (at w-face CV)	13
3.3 Temporal Discretization	14
3.4 Pressure Correction Method (Projection)	15
3.5 Poisson Solver	15
3.6 Verification of the Poisson Solver	18
3.7 Ghost Cells and Boundary Implementation	19
4 Implementation	21
4.1 Code Structure	21
4.2 Algorithm Flowchart	23
4.3 Key Implementation Details	23
5 Analytical Solution	25
5.1 Velocity Profile (Hagen–Poiseuille)	25
5.2 Temperature Profile	25
5.3 Validity of the Fully-Developed Temperature Profile	25
5.4 Nusselt Number	26

6	Results	28
6.1	Simulation Parameters	28
6.2	Convergence History	28
6.3	Velocity Field	29
6.4	Temperature Field	30
6.5	Velocity Profiles at Selected Axial Positions	31
6.6	Temperature Profiles at Selected Axial Positions	32
6.7	Comparison with Analytical Solution	33
7	Conclusion	34
A	2D Incompressible Flow in Cylindrical Coordinates (r, z) — Velocities (u, w)	35
A.1	Governing Equations (Strong Form)	35
A.2	Staggered Grid Layout	36
A.3	Finite Volume Discretisation — Integration over $\Delta r \Delta z$	36
A.3.1	Continuity Equation	36
A.3.2	Axial Momentum: CV at $(i, j + \frac{1}{2})$	36
A.3.3	Radial Momentum: CV at $(i + \frac{1}{2}, j)$	37
A.4	Projection Method	37
A.4.1	Step 1 — Predictor	37
A.4.2	Step 2 — Poisson Equation for Pressure Correction p'	38
A.4.3	Step 3 — Velocity Correction	38
A.4.4	Step 4 — Pressure Update	38
A.5	Poisson Solver Procedure	38
A.6	Ghost Cells and Boundary Conditions	39
B	Temperature Formula and Nusselt Number Audit	40
B.1	The Two Candidate Fully Developed Profiles	40
B.2	Governing Equation and Axial Gradient	40
B.3	Velocity Profile and Wall Boundary Condition	41
B.4	Integration of the Radial Equation	41
B.5	Integration Constant from the Global Energy Balance	41
B.6	Nusselt Number of the Corrected Profile	42
B.7	Why the Value is Exactly $48/11$	42
B.8	Assignment-Literal Profile: $Nu = 24/11$	43
B.9	Summary	44
C	MATLAB Source Code	45
C.1	Configuration and Initialisation	45
C.2	Main Simulation Script	45
C.3	Grid Generation	51
C.4	Boundary Conditions	51
C.5	Momentum Solver	52
C.6	Poisson Solver and Pressure Correction	58
C.7	Temperature Solver	68
C.8	Convergence Monitoring	70
C.9	Analytical Reference Solutions	71
C.10	Plotting Helper	71

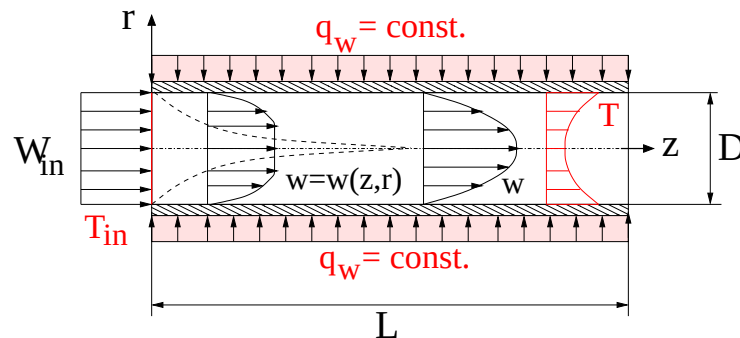
Assignment

Numerical Simulation and Modelling of Incompressible Flow

LV-Nr.: 321.053 **EX**, WS 2025/26

Entwicklung der Strömung am Eintritt eines zylindrischen Rohrs

Es soll eine zwei-dimensionale laminare Strömung in einem zylindrischen Rohr mit gegebenem Durchmesser D simuliert werden. Die gesamte Rohrwand wird mit konstanter Wärmestromdichte $q_w = \text{const.}$ beheizt. Das inkompressible Fluid habe gegebene konstante Dichte $\rho = \text{const.}$, dynamische Viskosität $\mu = \text{const.}$, Wärmeleitfähigkeit $\lambda = \text{const.}$, und spezifische Wärmekapazität $c_v = \text{const.}$. Viskose Dissipation kann in der thermischen Energiebilanz vernachlässigt werden. Die axiale Länge des Rohrs sei $L = 50D$. Die Strömung tritt mit einem gleichförmigen Geschwindigkeits- und Temperaturprofil $w|_{z=0} = W_{in} = \text{const.}$ bzw. $T|_{z=0} = T_{in} = \text{const.}$ in das Rohr ein. Das Strömungsgebiet ist unten stehend schematisch skizziert.



Aufgaben:

- Formulieren Sie die Erhaltungsgleichungen, welche dieses zwei-dimensionale Strömungsproblem beschreiben, in Zylinderkoordinaten.
- Entdimensionalisieren Sie die Bewegungsgleichungen unter Verwendung von D , W_{in} , ρW_{in}^2 , und $q_w / \rho c_v W_{in}$ als Bezugsgrößen für Länge, Geschwindigkeit, Druck, und Temperaturdifferenz $T - T_{in}$.
- Diskretisieren Sie die dimensionslosen Gleichungen
 - räumlich mittels Finite-Volumen Methode zweiter Ordnung genau
 - zeitlich mit einem expliziten Verfahren zweiter Ordnung genau.
- Programmieren Sie ein geeignetes direktes Lösungsverfahren zur Lösung der Poisson Gleichung, wie im Rahmen der Druckkorrektur zur Erfüllung der Kontinuitätsgleichung erforderlich.
- Simulieren Sie die Strömung für den Fall mit $Re = \rho W_{in} D / \mu = 100$, $Pr = \mu c_v / \lambda = 5$ bis das Strömungsfeld einen konvergierten Zustand erreicht hat.
- Dokumentieren Sie Ihre Arbeit. Bei der Darstellung der Ergebnisse sollen insbesondere gezeigt werden:
 - Kontouren der dimensionslosen Geschwindigkeitsfelder in beide Richtungen, also von w/W_{in} (axial) und u/W_{in} (radial), sowie Temperaturfelder $(T(x, y) - T_{in}) \rho W_{in} c_v / q_w$.
 - Profile von $w(r, z) / W_{in}$ und $(T(x, y) - T_{in}) \rho W_{in} c_v / q_w$ an ausgewählten Stromab-Positionen $z_i = \text{const.}$

Korrigierte Seite 2. Seite 1 ist unverändert. Auf dieser Seite ist *nur* die voll entwickelte Vergleichs-Temperaturformel berichtigt: in der Originalfassung fehlte der Faktor $\frac{1}{2}$ vor $Re Pr$ in den radialen Termen, und die additive Konstante muss $-\frac{7}{48}$ statt $-\frac{3}{4}$ lauten, damit die Mischtemperatur $\theta_{\text{bulk}}(z) = 4z/D$ für alle $z \geq 0$ korrekt ist. Die unveränderte Originalfassung liegt als **Assignment_WS25_error.pdf** bei; Herleitung und Literaturquellen ($Nu = 48/11$) sind im begleitenden Bericht angegeben.

- Fügen Sie zum Vergleich die Profile für die voll entwickelte zylindrische Rohrströmung hinzu, hier gegeben als

$$w = 2 W_{\text{in}} \left[1 - \left(\frac{2r}{D} \right)^2 \right],$$

$$\frac{(T - T_{\text{in}}) \varrho W_{\text{in}} c_v}{q_w} = 4 \frac{z}{D} + Re Pr \left[\frac{1}{2} \left(\frac{2r}{D} \right)^2 - \frac{1}{8} \left(\frac{2r}{D} \right)^4 - \frac{7}{48} \right],$$

welche für zunehmende Lauflänge z schlussendlich erreicht werden müssen.

Äquivalente Schreibweise (Faktor ausgeklammert):

$$\frac{(T - T_{\text{in}}) \varrho W_{\text{in}} c_v}{q_w} = 4 \frac{z}{D} + \frac{1}{2} Re Pr \left[\left(\frac{2r}{D} \right)^2 - \frac{1}{4} \left(\frac{2r}{D} \right)^4 - \frac{7}{24} \right].$$

Statement on the Use of AI Tools

AI-based tools (an LLM coding assistant, Claude) were used while preparing both the MATLAB solver and this report: for drafting and debugging portions of the MATLAB code, for typesetting and organising sections of the report and several of its schematic figures, and for wording and proofreading assistance.

Every governing equation, discretisation choice, boundary condition, and numerical result presented here was worked through, checked, and is understood by the author. The AI tool was used as an implementation and editing aid – to speed up typing, catch mistakes, and keep the document consistent – not as a substitute for the underlying engineering judgement: the derivation of the equations, the finite-volume discretisation and projection method, the choice of numerical schemes, and the interpretation of the results were all reasoned through and can be explained and defended independently of the tool, as required at the individual oral assessment.

1 Problem Description

1.1 Geometry and Setup

The problem considers two-dimensional laminar flow in a cylindrical pipe with diameter D and length $L = 50D$. The pipe wall is heated with constant heat flux $q_w = \text{const.}$. The incompressible fluid has constant properties: density ρ , dynamic viscosity μ , thermal conductivity λ , and specific heat capacity c_v . Viscous dissipation is neglected.

The flow enters with uniform velocity and temperature profiles:

$$w|_{z=0} = W_{\text{in}} = \text{const.} \quad (1)$$

$$T|_{z=0} = T_{\text{in}} = \text{const.} \quad (2)$$

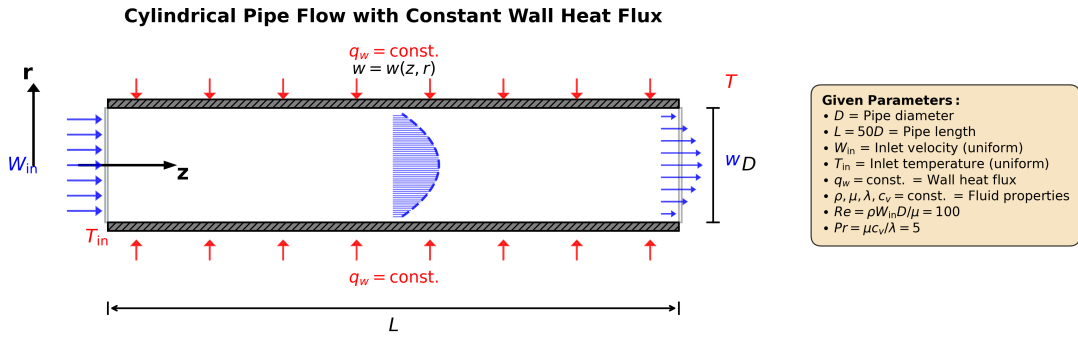


Figure 1: Schematic of the cylindrical pipe flow with constant wall heat flux. Flow enters with uniform velocity W_{in} and temperature T_{in} , and develops into a parabolic profile along the pipe length.

1.2 Physical Parameters

- Reynolds number: $Re = \frac{\rho W_{\text{in}} D}{\mu} = 100$
- Prandtl number: $Pr = \frac{\mu c_v}{\lambda} = 5$
- Length-to-diameter ratio: $L/D = 50$

2 Mathematical Formulation

2.1 Governing Equations in Cylindrical Coordinates

For axisymmetric flow in cylindrical coordinates (r, z) with velocity components (u, w) in the radial and axial directions, the governing equations in **conservation form** are:

2.1.1 Continuity Equation

$$\underbrace{\frac{1}{r} \frac{\partial(r u)}{\partial r}}_{\text{radial mass-flux divergence}} + \underbrace{\frac{\partial w}{\partial z}}_{\text{axial mass-flux divergence}} = 0 \quad (3)$$

The $1/r$ weighting on the radial term accounts for the circumferential area $2\pi r$ growing with radius; the two terms summing to zero enforces incompressibility (no net accumulation of mass in any control volume).

2.1.2 Momentum Equations

Radial momentum:

$$\frac{\partial u}{\partial t} + \frac{1}{r} \frac{\partial(r u^2)}{\partial r} + \frac{\partial(u w)}{\partial z} = -\frac{1}{\rho} \frac{\partial p}{\partial r} + \frac{1}{\rho} \left[\frac{1}{r} \frac{\partial(r \tau_{rr})}{\partial r} - \frac{\tau_{\varphi\varphi}}{r} + \frac{\partial \tau_{rz}}{\partial z} \right] \quad (4)$$

Axial momentum:

$$\frac{\partial w}{\partial t} + \frac{1}{r} \frac{\partial(r u w)}{\partial r} + \frac{\partial(w^2)}{\partial z} = -\frac{1}{\rho} \frac{\partial p}{\partial z} + \frac{1}{\rho} \left[\frac{1}{r} \frac{\partial(r \tau_{rz})}{\partial r} + \frac{\partial \tau_{zz}}{\partial z} \right] \quad (5)$$

Term by term, each momentum equation reads (local rate of change) + (convection) = (pressure force) + (viscous force):

- $\frac{\partial u}{\partial t}, \frac{\partial w}{\partial t}$ — **local (unsteady) acceleration**: the rate of change of momentum stored in a fixed control volume.
- $\frac{1}{r} \frac{\partial(r u^2)}{\partial r}, \frac{\partial(u w)}{\partial z}$ (and the analogous axial terms) — **convective transport**: momentum carried in and out of the control volume by the flow itself (advection by u and w).
- $-\frac{1}{\rho} \frac{\partial p}{\partial r}, -\frac{1}{\rho} \frac{\partial p}{\partial z}$ — **pressure-gradient force** per unit mass, the driving force pushing fluid from high to low pressure.
- $\frac{1}{\rho} \left[\frac{1}{r} \frac{\partial(r \tau_{rr})}{\partial r} + \frac{\partial \tau_{rz}}{\partial z} \right]$ and the axial counterpart — **viscous (diffusive) force** per unit mass, the net effect of internal friction spreading momentum from fast- to slow-moving fluid.
- $-\frac{\tau_{\varphi\varphi}}{\rho r}$ — a **curvature (“hoop-stress”) term** unique to cylindrical coordinates: it arises because the circumferential stress $\tau_{\varphi\varphi}$ acts on a surface whose area changes with r ; there is no Cartesian analogue.

The stress tensor components for a Newtonian fluid are:

$$\underbrace{\tau_{rr} = 2\mu \frac{\partial u}{\partial r}}_{\text{radial normal stress}}, \quad \underbrace{\tau_{\varphi\varphi} = 2\mu \frac{u}{r}}_{\text{hoop (circumferential) normal stress}}, \quad \underbrace{\tau_{zz} = 2\mu \frac{\partial w}{\partial z}}_{\text{axial normal stress}}, \quad \underbrace{\tau_{rz} = \mu \left(\frac{\partial u}{\partial z} + \frac{\partial w}{\partial r} \right)}_{\text{shear stress}} \quad (6)$$

2.1.3 Energy Equation

$$\underbrace{\frac{\partial T}{\partial t}}_{\text{local rate of change}} + \underbrace{\frac{1}{r} \frac{\partial(r u T)}{\partial r} + \frac{\partial(w T)}{\partial z}}_{\text{convective transport of heat by the flow}} = \underbrace{\frac{\lambda}{\rho c_v} \left[\frac{1}{r} \frac{\partial}{\partial r} \left(r \frac{\partial T}{\partial r} \right) + \frac{\partial^2 T}{\partial z^2} \right]}_{\text{conductive (diffusive) transport, Fourier's law}} \quad (7)$$

Viscous dissipation (heat generated by internal friction) is neglected, per the problem statement.

2.2 Boundary Conditions

Inlet ($z = 0$):

$$u = 0, \quad w = W_{\text{in}}, \quad T = T_{\text{in}} \quad (8)$$

Wall ($r = D/2$):

$$u = 0, \quad w = 0, \quad \lambda \frac{\partial T}{\partial r} \Big|_{r=D/2} = q_w \quad (9)$$

Heat flows into the fluid at the wall (constant heating), so with r increasing outward this is a positive gradient: the wall is hotter than the centreline.

Centreline ($r = 0$, symmetry):

$$u = 0, \quad \frac{\partial w}{\partial r} \Big|_{r=0} = 0, \quad \frac{\partial T}{\partial r} \Big|_{r=0} = 0 \quad (10)$$

Outlet ($z = L$, zero-gradient):

$$\frac{\partial u}{\partial z} = 0, \quad \frac{\partial w}{\partial z} = 0, \quad \frac{\partial T}{\partial z} = 0 \quad (11)$$

2.3 Non-Dimensionalization

2.3.1 Reference Scales

- Length: D
- Velocity: W_{in}
- Pressure: ρW_{in}^2
- Temperature difference: $\frac{q_w}{\rho c_v W_{\text{in}}}$
- Time: D/W_{in}

2.3.2 Non-Dimensional Variables

$$r^* = \frac{r}{D}, \quad z^* = \frac{z}{D}, \quad t^* = \frac{t W_{\text{in}}}{D} \quad (12)$$

$$u^* = \frac{u}{W_{\text{in}}}, \quad w^* = \frac{w}{W_{\text{in}}}, \quad p^* = \frac{p}{\rho W_{\text{in}}^2} \quad (13)$$

$$\theta = \frac{(T - T_{\text{in}}) \rho c_v W_{\text{in}}}{q_w} \quad (14)$$

2.3.3 Non-Dimensional Governing Equations

Dropping asterisks for brevity (all quantities are non-dimensional hereafter):

Continuity:

$$\frac{1}{r} \frac{\partial(r u)}{\partial r} + \frac{\partial w}{\partial z} = 0 \quad (15)$$

Radial momentum:

$$\frac{\partial u}{\partial t} + \frac{1}{r} \frac{\partial(r u^2)}{\partial r} + \frac{\partial(u w)}{\partial z} = -\frac{\partial p}{\partial r} + \frac{1}{\text{Re}} \left[\frac{1}{r} \frac{\partial}{\partial r} \left(r \frac{\partial u}{\partial r} \right) - \frac{u}{r^2} + \frac{\partial^2 u}{\partial z^2} \right] \quad (16)$$

Axial momentum:

$$\frac{\partial w}{\partial t} + \frac{1}{r} \frac{\partial(r u w)}{\partial r} + \frac{\partial(w^2)}{\partial z} = -\frac{\partial p}{\partial z} + \frac{1}{\text{Re}} \left[\frac{1}{r} \frac{\partial}{\partial r} \left(r \frac{\partial w}{\partial r} \right) + \frac{\partial^2 w}{\partial z^2} \right] \quad (17)$$

Energy:

$$\frac{\partial \theta}{\partial t} + \frac{1}{r} \frac{\partial(r u \theta)}{\partial r} + \frac{\partial(w \theta)}{\partial z} = \frac{1}{\text{RePr}} \left[\frac{1}{r} \frac{\partial}{\partial r} \left(r \frac{\partial \theta}{\partial r} \right) + \frac{\partial^2 \theta}{\partial z^2} \right] \quad (18)$$

2.3.4 Non-Dimensional Boundary Conditions

Inlet ($z = 0$): $u = 0, w = 1, \theta = 0$

Wall ($r = 1/2$): $u = 0, w = 0, \partial \theta / \partial r = \text{RePr}$

Non-dimensionalizing the wall condition directly: $\partial \theta / \partial r^* = D \cdot (\rho c_v W_{\text{in}} / q_w) \cdot \partial T / \partial r|_{\text{wall}} = D \cdot (\rho c_v W_{\text{in}} / q_w) \cdot (q_w / \lambda) = \rho c_v W_{\text{in}} D / \lambda = \text{RePr}$ (i.e. 500 for $\text{Re} = 100, \text{Pr} = 5$), *not* ± 1 — the reference length/temperature scales do not cancel.

Centreline ($r = 0$): $u = 0, \partial w / \partial r = 0, \partial \theta / \partial r = 0$

Outlet ($z = 50$): $\partial u / \partial z = 0, \partial w / \partial z = 0, \partial \theta / \partial z = 0$

3 Numerical Solution

3.1 Computational Domain and Grid

The computational domain is:

$$\Omega = \{(r, z) : 0 \leq r \leq 0.5, \quad 0 \leq z \leq 50\} \quad (19)$$

A **staggered grid** (MAC grid) is used:

- Scalar variables (p, θ) : cell centres $(r_{c,i}, z_{c,j})$
- Radial velocity u : radial faces $(r_{f,i}, z_{c,j})$
- Axial velocity w : axial faces $(r_{c,i}, z_{f,j})$

Grid parameters: $n_r = 50$ cells radially, $n_z = 250$ cells axially, giving:

$$\Delta r = \frac{0.5}{n_r} = 0.01, \quad \Delta z = \frac{50}{n_z} = 0.2 \quad (20)$$

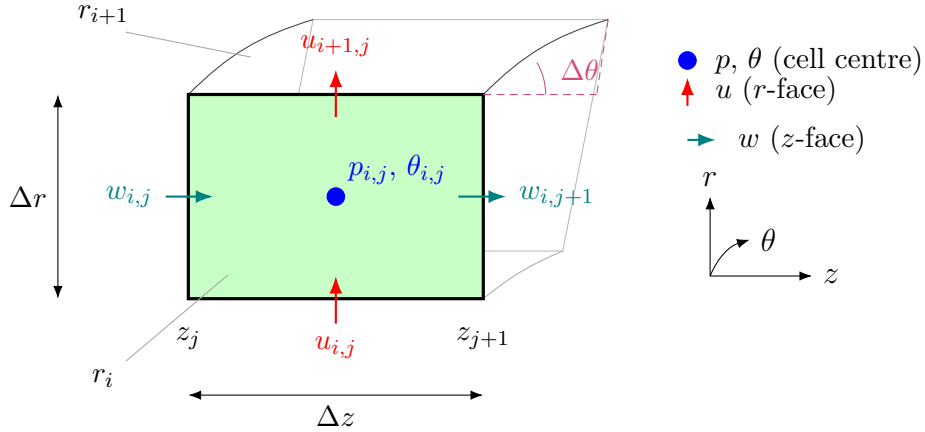


Figure 2: Staggered grid in the (r, z) plane. Scalars (p, θ) at the cell centre $(r_{c,i}, z_{c,j})$ (blue); radial velocity u on the (curved, cylindrical) r -faces (red), stored at $u_{i,j}$ and $u_{i+1,j}$; axial velocity w on the z -faces (teal), stored at $w_{i,j}$ and $w_{i,j+1}$. The control volume is drawn swept through a finite $\Delta\theta$ purely to convey the 3-D cylindrical shape of the r -faces (note the outer face at r_{i+1} sweeps further than the inner face at r_i , since arc length $= r \Delta\theta$); the axisymmetric problem itself has $\partial/\partial\theta = 0$ and no θ -dependence.

3.2 Spatial Discretization: Finite Volume Method

The governing equations are integrated over each control volume using the FVM. After applying Gauss' divergence theorem, the volume integrals become surface flux integrals that are evaluated at control-volume faces.

3.2.1 Continuity (at scalar CV)

$$\frac{r_{i+\frac{1}{2}} u_{i+1,j} - r_{i-\frac{1}{2}} u_{i,j}}{r_{c,i} \Delta r} + \frac{w_{i,j+1} - w_{i,j}}{\Delta z} = 0 \quad (21)$$

3.2.2 Convective Scheme: 2nd-Order Upwind (Linear Upwind Differencing)

What an upwind scheme is. In the finite volume method the transported quantity ϕ (w , u or θ) is stored at cell centres, but the convective flux $F = v \phi$ has to be evaluated at the cell *faces*, which lie between two centres. The face value ϕ_{face} must therefore be interpolated from neighbouring cells, and the choice of this interpolation is the convective scheme.

The obvious choice is **central differencing**, the average of the two neighbours,

$$\phi_{\text{face}} = \frac{1}{2} (\phi_i + \phi_{i+1}). \quad (22)$$

It is second-order accurate but treats both sides of the face equally. Convection, however, is directional: information is carried *with* the flow, so the value arriving at a face comes from the upstream side. When convection dominates diffusion, central differencing produces non-physical oscillations (over- and undershoots). The governing parameter is the cell Peclet number, the ratio of convective to diffusive transport across one cell,

$$\text{Pe} = \frac{|v| \Delta x}{\nu_{\text{eff}}}, \quad (23)$$

and central differencing only remains bounded for $\text{Pe} < 2$. For our grid ($\Delta z = 0.2$, $\Delta r = 0.01$), based on the mean axial velocity $|w| = 1$,

$$\text{Pe}_z = |w| \Delta z \text{Re} = 1 \times 0.2 \times 100 = 20 \gg 2 \quad (24)$$

and $\text{Pe}_\theta = |w| \Delta z \text{RePr} = 100$ for the energy equation. On the centreline, where $w \approx 2$, these values double to 40 and 200. Pure central differencing is therefore unsuitable for the convective terms of this problem.

Upwind schemes instead take the face value from the side the flow comes from, as decided by the sign of the transporting velocity at the face:

- **1st-order upwind:** $\phi_{\text{face}} = \phi_i$ if $v_{\text{face}} > 0$ (flow from cell i towards $i+1$), otherwise $\phi_{\text{face}} = \phi_{i+1}$. It is very robust and does not oscillate, but it is only first-order accurate and introduces artificial (numerical) diffusion that smears gradients.
- **2nd-order upwind (linear upwind differencing, LUD),** used in this work: the face value is linearly extrapolated from the *two* upstream cells.

For a generic flux $F = v \phi$ at the face between cells i and $i+1$, LUD reads

$$\phi_{\text{face}} = \begin{cases} \frac{3}{2}\phi_i - \frac{1}{2}\phi_{i-1} & \text{if } v_{\text{face}} > 0 \quad (\text{upstream} = \text{low-index side}) \\ \frac{3}{2}\phi_{i+1} - \frac{1}{2}\phi_{i+2} & \text{if } v_{\text{face}} < 0 \quad (\text{upstream} = \text{high-index side}) \end{cases} \quad (25)$$

which reduces to the 1st-order upwind value (ϕ_i or ϕ_{i+1}) if the linear extrapolation term is dropped. Equation (25) is implemented in `upwind2_face.m` (Listing 11) and is called by all three right-hand-side functions `compute_rhs_w.m`, `compute_rhs_u.m` and `compute_rhs_T.m`. LUD keeps the upstream bias needed for stability at $\text{Pe} \gg 2$, while on a uniform grid and for a smooth solution its truncation error is $\mathcal{O}(\Delta x^2)$ (shown below), so the convective discretization is formally second-order accurate, as required by the assignment. Diffusive terms retain standard **second-order central differencing**, since diffusion has no preferred direction.

Trade-off: LUD is not bounded. Second-order accuracy is not a boundedness guarantee. Because the extrapolated face value is not limited, LUD can overshoot or undershoot near sharp transients, boundaries, or insufficiently resolved gradients; the scheme is second-order in smooth regions but is not monotone or TVD everywhere. In the present simulation this appears only as a small, localized, transient undershoot of θ near the inlet-wall corner during the first time steps (Fig. 11); on coarser grids the undershoot would be larger. Bounded alternatives are TVD schemes with flux limiters (e.g. minmod or van Leer), which blend LUD with 1st-order upwind wherever the solution gradient is steep.

Why these reconstructions are second order. Expand ϕ in a Taylor series about the face location x_f (face between cells i and $i+1$, spacing Δx , upwind case $v_{\text{face}} > 0$): with $x_i = x_f - \frac{1}{2}\Delta x$ and $x_{i-1} = x_f - \frac{3}{2}\Delta x$,

$$\phi_i = \phi_f - \frac{1}{2}\Delta x \phi'_f + \frac{1}{8}\Delta x^2 \phi''_f + \mathcal{O}(\Delta x^3), \quad \phi_{i-1} = \phi_f - \frac{3}{2}\Delta x \phi'_f + \frac{9}{8}\Delta x^2 \phi''_f + \mathcal{O}(\Delta x^3). \quad (26)$$

Plain 1st-order upwind ($\phi_{\text{face}} = \phi_i$) retains the $-\frac{1}{2}\Delta x \phi'_f$ term and is therefore only $\mathcal{O}(\Delta x)$ accurate. Forming the linear-upwind combination $\frac{3}{2}\phi_i - \frac{1}{2}\phi_{i-1}$ cancels the ϕ'_f term *exactly* ($-\frac{3}{2} \cdot \frac{1}{2}\Delta x + \frac{1}{2} \cdot \frac{3}{2}\Delta x = 0$), leaving $\phi_{\text{face}} = \phi_f - \frac{3}{8}\Delta x^2 \phi''_f + \mathcal{O}(\Delta x^3)$: the leading error is $\mathcal{O}(\Delta x^2)$, confirming LUD is second order. The central-difference diffusion operator is second order by the same mechanism applied symmetrically: expanding $\phi_{i\pm 1}$ about x_i and summing, the odd-order ($\Delta x, \Delta x^3$) terms cancel by symmetry, leaving $(\phi_{i+1} - 2\phi_i + \phi_{i-1})/\Delta x^2 = \phi''_i + \frac{1}{12}\Delta x^2 \phi''''_i + \mathcal{O}(\Delta x^4)$.

3.2.3 Temperature equation and cell-by-cell development

The temperature variable $\theta_{i,j}$ is stored at the centre of every scalar control volume. At each time step, its value is advanced from the net transport and diffusion through the four faces of that control volume:

$$\frac{d\theta_{i,j}}{dt} = -\frac{1}{r_{c,i}\Delta r} [(ru\theta)_e - (ru\theta)_w] - \frac{(w\theta)_n - (w\theta)_s}{\Delta z} + \frac{1}{\text{RePr}} \left[\frac{r_{i+\frac{1}{2}}\theta_r|_e - r_{i-\frac{1}{2}}\theta_r|_w}{r_{c,i}\Delta r} + \frac{\theta_z|_n - \theta_z|_s}{\Delta z} \right]. \quad (27)$$

The convective face temperatures are reconstructed with the LUD formula above, while the diffusive face gradients use central differences. The wall face receives the prescribed heat flux through the ghost value $\theta_g = \theta_{i,j} + \Delta r \text{RePr}$; the inlet face uses $\theta = 0$ and the outlet uses a zero axial gradient. Thus every cell is updated from its local radial and axial neighbour values, together with the local face velocities. In the AB2 time integrator the resulting cellwise right-hand side is stored and updated as

$$\theta_{i,j}^{n+1} = \theta_{i,j}^n + \Delta t \left(\frac{3}{2}R_{\theta,i,j}^n - \frac{1}{2}R_{\theta,i,j}^{n-1} \right), \quad (28)$$

with a forward-Euler startup step when the previous right-hand side is unavailable. Heat therefore first enters cells adjacent to the wall and then spreads radially and downstream; the centreline and downstream cells develop later because they receive heat through diffusion and axial advection rather than directly from the heated wall.

3.2.4 Axial Momentum (at w-face CV)

The w-CV at $(r_{c,i}, z_{f,j})$ spans $[r_{f,i}, r_{f,i+1}] \times [z_{c,j-1}, z_{c,j}]$.

AB2 predictor step (including the known pressure gradient):

$$\boxed{w_{i,j}^* = w_{i,j}^n + \Delta t \left[\frac{3}{2} F_{w,i,j}^n - \frac{1}{2} F_{w,i,j}^{n-1} - \frac{p_{i,j}^n - p_{i,j-1}^n}{\Delta z} \right], \quad F_w = -C_r^w - C_z^w + D_r^w + D_z^w} \quad (29)$$

where the convective terms (upwind) are:

$$C_r^w = \frac{r_{i+\frac{1}{2}} u_E w_{E,\text{up}} - r_{i-\frac{1}{2}} u_I w_{I,\text{up}}}{r_{c,i} \Delta r} \quad (30)$$

$$C_z^w = \frac{\bar{w}_N w_{N,\text{up}} - \bar{w}_S w_{S,\text{up}}}{\Delta z} \quad (31)$$

and the diffusive terms (central) are:

$$D_r^w = \frac{1}{\text{Re}} \frac{r_{i+\frac{1}{2}} (w_{i+1,j} - w_{i,j}) - r_{i-\frac{1}{2}} (w_{i,j} - w_{i-1,j})}{r_{c,i} \Delta r^2} \quad (32)$$

$$D_z^w = \frac{1}{\text{Re}} \frac{w_{i,j+1} - 2w_{i,j} + w_{i,j-1}}{\Delta z^2} \quad (33)$$

The radial momentum equation is discretized analogously at u-face control volumes.

3.3 Temporal Discretization

Explicit second-order Adams–Bashforth time stepping is used. For a spatial operator F , the update is

$$F^{AB2} = \frac{3}{2} F^n - \frac{1}{2} F^{n-1}, \quad (34)$$

$$\phi^{n+1} = \phi^n + \Delta t F^{AB2}. \quad (35)$$

The first step uses forward Euler because F^{n-1} is not yet available. The current momentum and temperature RHS arrays are stored for the following step. AB2 needs one RHS evaluation and one pressure-Poisson solve per time step.

Why AB2 is second order. Forward Euler, $\phi^{n+1} = \phi^n + \Delta t F^n$, freezes the right-hand side at its value at t^n and so approximates the average slope over $[t^n, t^{n+1}]$ with $\mathcal{O}(\Delta t)$ error, giving local truncation error $\mathcal{O}(\Delta t^2)$ per step and global error $\mathcal{O}(\Delta t)$. AB2 instead linearly extrapolates the two most recent slopes. Expanding $F^{n-1} = F(t^n - \Delta t) = F^n - \Delta t \dot{F}^n + \frac{1}{2} \Delta t^2 \ddot{F}^n + \mathcal{O}(\Delta t^3)$,

$$\frac{3}{2} F^n - \frac{1}{2} F^{n-1} = F^n + \frac{1}{2} \Delta t \dot{F}^n - \frac{1}{4} \Delta t^2 \ddot{F}^n + \mathcal{O}(\Delta t^3), \quad (36)$$

which matches the true average slope on $[t^n, t^{n+1}]$, $F^n + \frac{1}{2} \Delta t \dot{F}^n + \mathcal{O}(\Delta t^2)$, through the $\mathcal{O}(\Delta t)$ term: the leading error in the slope estimate is pushed from $\mathcal{O}(\Delta t^0)$ (Euler) to $\mathcal{O}(\Delta t^2)$. Multiplying by Δt in the update gives local truncation error $\mathcal{O}(\Delta t^3)$ per step; accumulating over the $N = \mathcal{O}(1/\Delta t)$ steps needed to reach a fixed final time gives global error $\mathcal{O}(\Delta t^2)$, i.e. AB2 is second order. Because the pressure-Poisson equation is solved directly (Section 3.5, no iterative truncation) at every step, it introduces no additional error beyond machine precision; combined with the $\mathcal{O}(\Delta x^2)$ spatial operators above, the overall scheme is therefore second order in *both* space and time, as the assignment requires.

The temperature equation uses the same AB2 extrapolation of its convection–diffusion operator.

The time step $\Delta t = 0.002$ satisfies the stability constraints:

- Convective CFL: $\Delta t / \Delta z = 0.01 \ll 1$
- Viscous stability: $\Delta t (1/\text{Re}) (1/\Delta r^2 + 1/\Delta z^2) \approx 0.2 < 0.5$

3.4 Pressure Correction Method (Projection)

A segregated approach is used. Each time step consists of:

1. **Predictor:** Extrapolate the convective–diffusive momentum RHS with AB2 and include the known pressure gradient:

$$\vec{u}^* = \vec{u}^n + \Delta t \left[\frac{3}{2} F^n - \frac{1}{2} F^{n-1} - \nabla p^n \right]. \quad (37)$$

2. **Poisson equation:** Find pressure correction p' such that the corrected velocity is divergence-free:

$$\frac{1}{r} \frac{\partial}{\partial r} \left(r \frac{\partial p'}{\partial r} \right) + \frac{\partial^2 p'}{\partial z^2} = \frac{1}{\Delta t} \left[\frac{1}{r} \frac{\partial (r u^*)}{\partial r} + \frac{\partial w^*}{\partial z} \right] \quad (38)$$

Discretized:

$$\frac{r_{i+\frac{1}{2}}(p'_{i+1,j} - p'_{i,j}) - r_{i-\frac{1}{2}}(p'_{i,j} - p'_{i-1,j})}{r_{c,i} \Delta r^2} + \frac{p'_{i,j+1} - 2p'_{i,j} + p'_{i,j-1}}{\Delta z^2} = b_{i,j} \quad (39)$$

3. **Velocity correction:**

$$u_{i,j}^{n+1} = u_{i,j}^* - \Delta t \frac{p'_{i,j} - p'_{i-1,j}}{\Delta r} \quad (40)$$

$$w_{i,j}^{n+1} = w_{i,j}^* - \Delta t \frac{p'_{i,j} - p'_{i,j-1}}{\Delta z} \quad (41)$$

4. **Pressure update** with under-relaxation:

$$p^{n+1} = p^n + \alpha_p p', \quad \alpha_p = 0.7 \quad (42)$$

3.5 Poisson Solver

The assignment requires a *direct* solution method for the pressure Poisson equation, and requires it to be *programmed* rather than delegated to a library routine. Accordingly no use is made of MATLAB’s backslash operator (`A\b`, i.e. the UMFPACK sparse LU), of `decomposition`, or of SciPy’s `spsolve`: the elimination is written out in `lu_factor.m`, `lu_solve.m`, `factorize_poisson.m` and `solve_poisson.m`. Backslash appears in exactly one file in the whole repository, the verification script `verify_poisson_solver.m`, where it serves as an independent reference (Sec. 3.6).

In the terminology of the course notes [5], Sec. III.3.1, the scheme used here combines two of the four direct solvers presented there: the Thomas algorithm (TDMA, method (C)), in its block form, as the outer elimination, with an LU decomposition (method (B)) as the kernel that factorises each block. The same section notes that the TDMA, although in itself a one-dimensional algorithm, “is often used as sub-algorithm of other solving methods for multidimensional, i.e. non-tridiagonal, elliptic systems” [5] — which is precisely the role it plays here. The notes present the TDMA for a scalar tridiagonal matrix; the block form used here, in which the scalar entries are replaced by the $n_r \times n_r$ blocks of Eq. (43), is the direct extension of that algorithm to the two-dimensional operator. The iterative alternatives of Sec. III.3.2 (Gauss–Seidel, SOR, LSOR, ADISOR) are excluded by the assignment’s requirement of a *direct* method.

Block structure. Group the $n_r \cdot n_z = 12,500$ unknowns by axial station j , so that column j of the field p' forms one block $\mathbf{p}_j$ of length n_r . The five-point stencil of Eq. (38) couples a cell only to its radial neighbours (same j) and to $j \pm 1$, so the operator is **block tridiagonal** with $n_z = 250$ blocks of size $n_r \times n_r = 50 \times 50$:

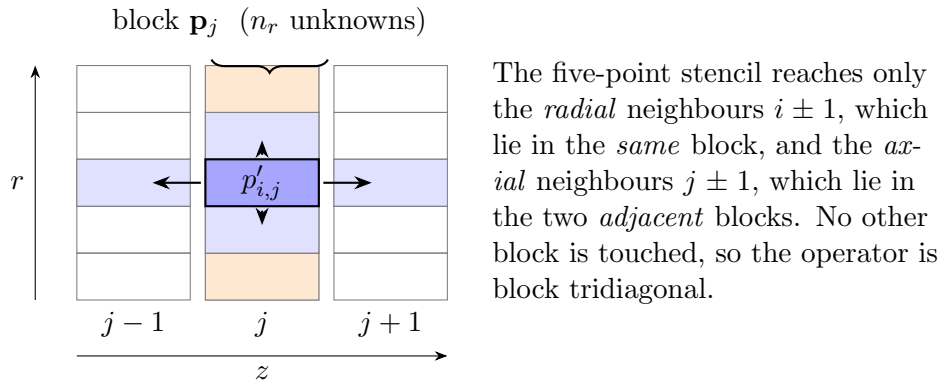
$$\mathbf{S}_j \mathbf{p}_{j-1} + \mathbf{D}_j \mathbf{p}_j + \mathbf{N}_j \mathbf{p}_{j+1} = \mathbf{b}_j, \quad j = 1, \dots, n_z, \quad (43)$$

with

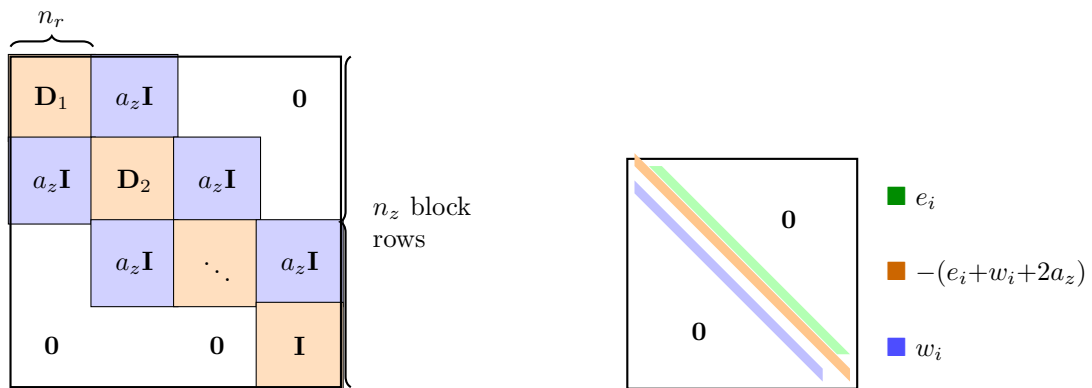
$$\mathbf{D}_j = \text{tridiag}(w_i, -(e_i + w_i + 2a_z), e_i), \quad \mathbf{S}_j = \mathbf{N}_j = a_z \mathbf{I}, \quad (44)$$

$$e_i = \frac{r_{i+\frac{1}{2}}}{r_i \Delta r^2}, \quad w_i = \frac{r_{i-\frac{1}{2}}}{r_i \Delta r^2}, \quad a_z = \frac{1}{\Delta z^2}. \quad (45)$$

The boundary conditions enter as modifications of these coefficients rather than as extra equations: $e_{n_r} = 0$ (Neumann at the wall); $w_1 = 0$ automatically, because $r_{\frac{1}{2}} = 0$ (Neumann at the centreline); no south coupling at $j = 1$, so that diagonal carries a_z once instead of twice (Neumann at the inlet); and $\mathbf{D}_{n_z} = \mathbf{I}$, $\mathbf{S}_{n_z} = \mathbf{N}_{n_z} = \mathbf{0}$ (Dirichlet reference at the outlet). Figure 3 sketches this structure.



(a) Grouping the unknowns by axial station. Column j of the $(n_r \times n_z)$ pressure-correction field is one block $\mathbf{p}_j$.



(b) Block-tridiagonal structure. The last block row carries the Dirichlet outlet condition $p' = 0$. (c) Each diagonal block $\mathbf{D}_j$ is itself tridiagonal ($n_r \times n_r$), from the radial stencil.

Figure 3: Structure of the discrete pressure-Poisson operator that the block Thomas algorithm exploits.

Block Thomas elimination. Equation (43) is eliminated by the block form of the Thomas algorithm. The forward elimination forms the Schur complements

$$\mathbf{M}_1 = \mathbf{D}_1, \quad \mathbf{M}_j = \mathbf{D}_j - \mathbf{S}_j \mathbf{M}_{j-1}^{-1} \mathbf{N}_{j-1} = \mathbf{D}_j - a_z^2 \mathbf{M}_{j-1}^{-1}, \quad (46)$$

after which the solution follows from two sweeps over the axial stations,

$$\mathbf{y}_j = \mathbf{M}_j^{-1} (\mathbf{b}_j - a_z \mathbf{y}_{j-1}), \quad j = 1, \dots, n_z \quad (\text{forward}), \quad (47)$$

$$\mathbf{p}_j = \mathbf{y}_j - a_z \mathbf{M}_j^{-1} \mathbf{p}_{j+1}, \quad j = n_z, \dots, 1 \quad (\text{back substitution}). \quad (48)$$

Each $\mathbf{M}_j$ is factorised with a hand-written Doolittle LU with partial pivoting, $\mathbf{P}\mathbf{M}_j = \mathbf{L}\mathbf{U}$ (`lu_factor.m`), and inverted by forward/back substitution against the identity (`lu_solve.m`). Because the unknowns are grouped by axial station, column j of the $(n_r \times n_z)$ field is block j of the system; the implementation therefore needs no flattening and no flat-index bookkeeping at all. Figure 4 shows the resulting algorithm.

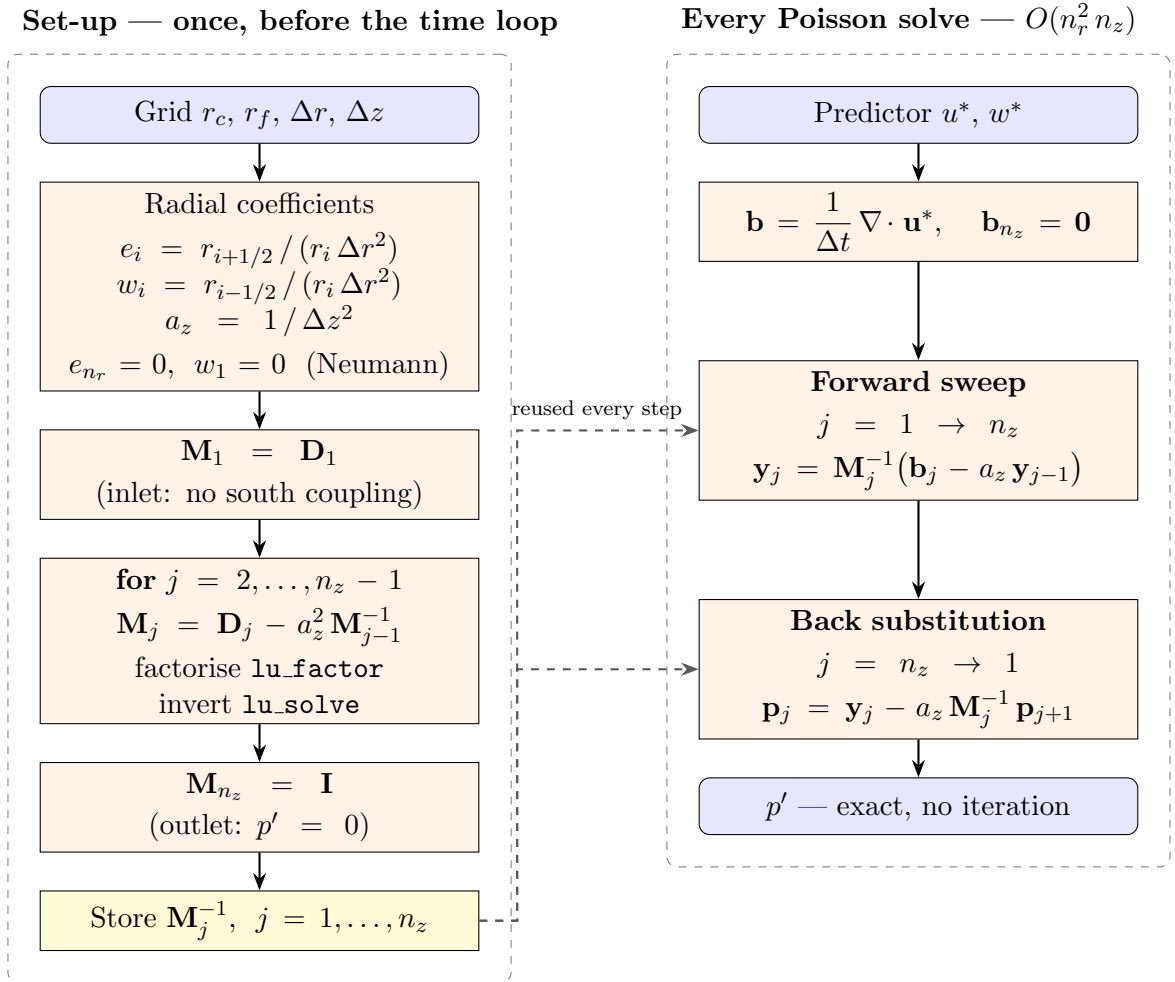


Figure 4: Block Thomas algorithm as implemented. The operator does not change in time, so the expensive elimination (left) runs once before the time loop; every pressure projection (right) is then two sweeps over the n_z axial stations, with a fixed operation count and no convergence test.

Cost. The operator is constant in time, so the factorisation (46) is performed once, before the time loop, at a cost of $O(n_z n_r^3) \approx 3 \times 10^7$ flops — 0.16s measured, in both

MATLAB and Python. This is the advantage of LU-type factorisation stressed in [5]: when the coefficient matrix does not change — as here, with a temporally constant mesh — the costly factorisation has to be carried out only once, and every subsequent solve needs only the cheap substitution steps. Each subsequent Poisson solve is then only the two sweeps above, i.e. two 50×50 matrix–vector products per axial station, $O(n_z n_r^2) \approx 1.2 \times 10^6$ flops. Measured: 1.8 ms per solve in MATLAB and 1.0 ms in Python, the same order as the backslash solve it replaces (0.5–0.9 ms) and negligible beside the momentum and energy right-hand-side evaluations.

Stability without pivoting between blocks. By construction the diagonal coefficient of Eq. (38) is minus the sum of the four neighbour coefficients, so $|a_{kk}| = \sum_{l \neq k} |a_{kl}|$ in every interior row, with strict inequality in the Dirichlet outlet rows. Hence $-\mathbf{A}$ is an irreducibly diagonally dominant M-matrix, a property inherited by every Schur complement $\mathbf{M}_j$ in Eq. (46). LU without pivoting is therefore guaranteed to exist and to be backward stable for this operator, which is what makes a hand-written Thomas-type elimination safe here; partial pivoting is retained inside `lu_factor.m` regardless, so that the kernel stays generally usable.

Boundary conditions for p' :

- Centreline ($r = 0$): Neumann $\partial p' / \partial r = 0$ (natural: $r_{-\frac{1}{2}} = 0$, flux vanishes)
- Wall ($r = 0.5$): Neumann $\partial p' / \partial r = 0$
- Inlet ($z = 0$): Neumann $\partial p' / \partial z = 0$
- Outlet ($z = 50$): Dirichlet $p' = 0$ is imposed at the last pressure cell centre as the reference. This location is half a cell upstream of the physical outlet face, so the condition is a discrete pressure reference rather than an exactly face-centred outlet condition.

3.6 Verification of the Poisson Solver

Because the elimination is written by hand, it is verified explicitly. `verify_poisson_solver.m` (and its Python twin `verify_poisson_solver.py`) applies the solver to a random, divergence-carrying velocity predictor on the production grid and compares it with the same system assembled independently as a sparse matrix by `build_poisson_matrix.m` — a routine that is otherwise unused by the time loop and is kept only for this check. Table 1 lists the outcome.

Table 1: Verification of the hand-written direct Poisson solver on the production grid ($n_r = 50$, $n_z = 250$, 12,500 unknowns). Relative residuals are $\|\mathbf{A}\mathbf{p}' - \mathbf{b}\|_\infty / \|\mathbf{b}\|_\infty$.

Quantity	MATLAB	Python
Dense LU kernel, 12×12 random system: $\ \mathbf{M}\mathbf{x} - \mathbf{b}\ _\infty$	1.2×10^{-14}	2.0×10^{-15}
Relative residual, block Thomas (this work)	1.7×10^{-13}	3.8×10^{-13}
Relative residual, reference library solver	4.6×10^{-14}	1.9×10^{-13}
$\ \mathbf{p}'_{\text{own}} - \mathbf{p}'_{\text{ref}}\ _\infty / \ \mathbf{p}'\ _\infty$	3.0×10^{-10}	4.4×10^{-10}
$\text{cond}_1(\mathbf{A})$ (estimate)	1.19×10^8	
$\text{cond}_1(\mathbf{A}) \cdot \varepsilon_{\text{mach}}$	2.6×10^{-8}	
$\max p' $ on the outlet column (imposed $p' = 0$)	0	0
Factorisation time (once, before the time loop)	159 ms	154 ms
Solve time (per Poisson solve)	1.80 ms	0.97 ms

The reference solver is MATLAB’s sparse backslash (UMFPACK) and SciPy’s `spsolve` (SuperLU) respectively. The meaningful accuracy measure is the *backward* error: the hand-written solver and the library solver reach the same relative residual, to within a factor of four. The solution-level difference between them, 3×10^{-10} , is not an error in either solver but a consequence of the operator’s conditioning — it lies two orders of magnitude *below* the bound $\text{cond}_1(\mathbf{A}) \varepsilon_{\text{mach}} = 2.6 \times 10^{-8}$, so the two are indistinguishable to the accuracy the discrete operator permits.

The substitution was also checked end to end: running 200 Heun steps (400 Poisson solves) with the library solver and with the block-Thomas solver, starting from identical initial conditions, leaves u , w , p and θ agreeing to a relative 1.6×10^{-12} or better. Replacing the library call by the hand-written direct solver therefore changes the computed flow field only at round-off level.

3.7 Ghost Cells and Boundary Implementation

Ghost cells are used to impose the velocity and temperature boundary conditions in the staggered grid. Pressure-correction boundary conditions are imposed directly in the block coefficients of the Poisson operator (Sec. 3.5); the outlet reference is applied at the last pressure cell centre rather than through a pressure ghost cell.

Table 2: Ghost cell values for each boundary.

Boundary	u_{ghost}	w_{ghost}	p_{ghost}	θ_{ghost}
Centreline ($r = 0$)	$u_g = 0$	$w_g = w_1$ (symmetry)	$p_g = p_1$	$\theta_g = \theta_1$
Wall ($r = 0.5$)	$u_g = 0$	$w_g = -w_{n_r}$ (no-slip)	$p_g = p_{n_r}$	$\theta_g = \theta_{n_r} + \Delta r \cdot \text{RePr}$
Inlet ($z = 0$)	$u_g = -u_1$	$w = 1$ (Dirichlet)	$p_g = p_1$	$\theta = 0$ (Dirichlet)
Outlet ($z = L$)	$u_g = u_{n_z}$	$w_g = w_{n_z}$	$p' = 0$	$\theta_g = \theta_{n_z}$

The wall temperature ghost cell $\theta_g = \theta_{n_r} + \Delta r \cdot \text{RePr}$ implements the Neumann condition $\partial\theta/\partial r = \text{RePr}$ via central differencing:

$$\left. \frac{\partial\theta}{\partial r} \right|_{\text{wall}} \approx \frac{\theta_g - \theta_{n_r}}{\Delta r} = \frac{\Delta r \cdot \text{RePr}}{\Delta r} = \text{RePr} \quad (49)$$

An earlier version of this derivation incorrectly dropped the RePr factor (giving $\partial\theta/\partial r = -1$). The full re-derivation and cross-checks are included in Appendix B.

4 Implementation

4.1 Code Structure

The solver is implemented in MATLAB. Table 3 lists all functions and their roles.

Table 3: MATLAB function structure.

Function	Description
<code>constants.m</code>	Returns struct with physical parameters (Re, Pr), grid sizes, time step, tolerances
<code>make_grid.m</code>	Generates staggered grid: $r_c, r_f, z_c, z_f, \Delta r, \Delta z$
<code>initialise_fields.m</code>	Allocates fields (u, w, p, θ) with uniform initial conditions
<code>apply_bc_w.m</code>	Enforces axial velocity BCs (inlet Dirichlet, outlet zero-gradient)
<code>apply_bc_u.m</code>	Enforces radial velocity BCs (axis and wall: $u = 0$)
<code>apply_bc_T.m</code>	Enforces temperature BCs (inlet Dirichlet, outlet zero-gradient)
<code>apply_all_bc.m</code>	Dispatches to <code>apply_bc_u/w/T.m</code> in one call (used for the initial-condition BCs before the time loop)
<code>upwind2_face.m</code>	2nd-order upwind (linear-extrapolation) face-value helper, shared by all three RHS functions below
<code>compute_rhs_w.m</code>	Explicit RHS of axial momentum (no pressure): 2nd-order upwind convection, central diffusion
<code>compute_rhs_u.m</code>	Explicit RHS of radial momentum (no pressure): 2nd-order upwind convection, central diffusion
<code>compute_rhs_T.m</code>	Explicit RHS of the energy equation: 2nd-order upwind convection, central diffusion
<code>predictor_w.m</code>	Thin 1-stage-Euler wrapper around <code>compute_rhs_w.m</code> (kept for standalone use)
<code>predictor_u.m</code>	Thin 1-stage-Euler wrapper around <code>compute_rhs_u.m</code> (kept for standalone use)
<code>build_poisson_matrix.m</code>	Assembles the same operator as a sparse matrix; used only by <code>verify_poisson_solver.m</code>
<code>lu_factor.m</code>	Hand-written Doolittle LU with partial pivoting, $\mathbf{PM} = \mathbf{LU}$ (elimination kernel)
<code>lu_solve.m</code>	Forward/back substitution for the factors from <code>lu_factor.m</code>
<code>factorize_poisson.m</code>	Block-Thomas factorisation of the block-tridiagonal Poisson operator (called once)
<code>solve_poisson.m</code>	Solves $\mathbf{A}\mathbf{p}' = \mathbf{b}$ by the two block-Thomas sweeps (direct, no backslash)
<code>verify_poisson_solver.m</code>	Verifies the hand-written solver against the assembled operator and a library reference
<code>correct_velocity.m</code>	Applies pressure-gradient correction to u^*, w^*
<code>update_pressure.m</code>	Updates pressure: $p^{n+1} = p^n + \alpha_p p'$
<code>advance_temperature.m</code>	Thin 1-stage-Euler wrapper around <code>compute_rhs_T.m</code> (kept for standalone use)
<code>ab2_step.m</code>	Advances (u, w, p, θ) with AB2 and one incremental pressure correction; stores the current RHS for the next step
<code>compute_divergence.m</code>	Computes discrete divergence of velocity field
<code>compute_residuals.m</code>	Computes convergence residuals ($R_{\text{cont}}, R_{\text{vel}}, R_\theta$)

4.2 Algorithm Flowchart

Figure 5 shows the algorithm implemented in `main.m`.

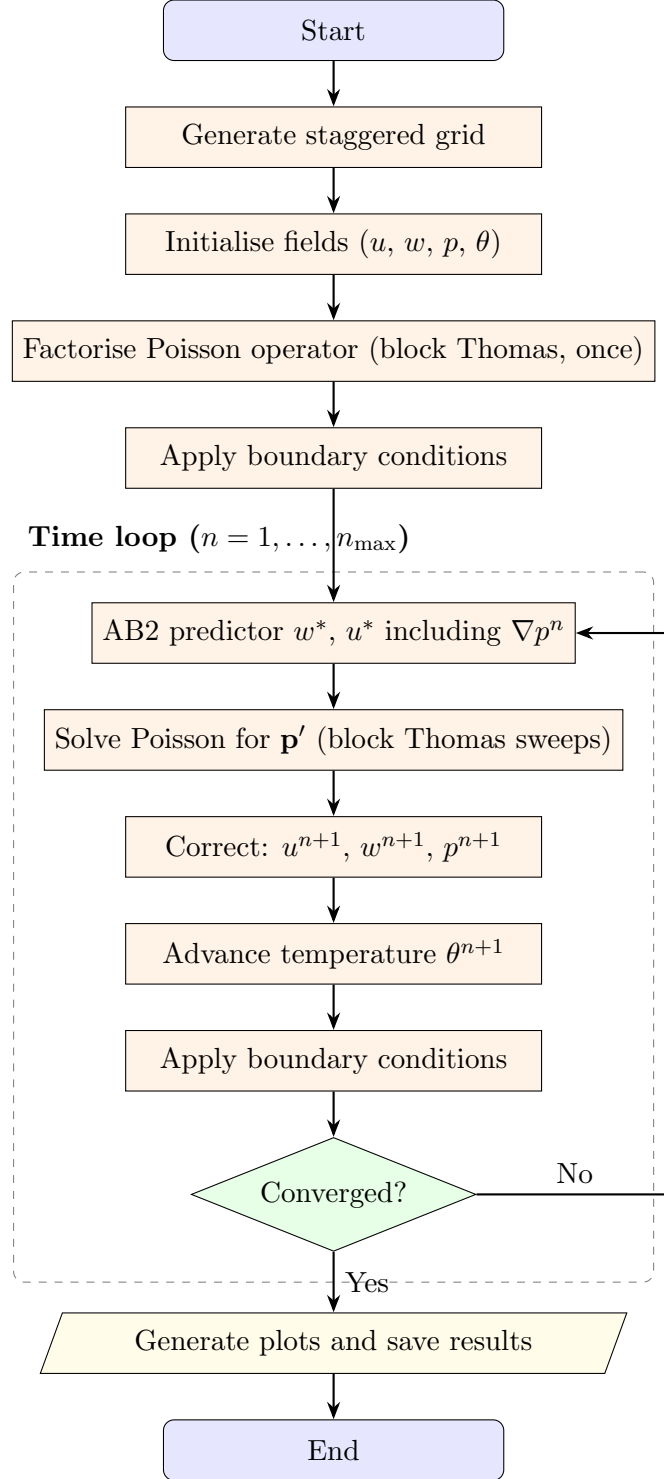


Figure 5: Algorithm flowchart for the pipe flow solver.

4.3 Key Implementation Details

- **2nd-order upwind differencing:** Convective fluxes in both momentum and energy equations use LUD. It is formally second-order on smooth, resolved regions,

as required by the assignment, but it is unbounded and can produce small local overshoots near sharp transients or boundaries.

- **Hand-written direct solver:** The Poisson operator (12,500 unknowns) is block tridiagonal — 250 blocks of 50×50 — and is eliminated by a block Thomas algorithm built on our own Doolittle LU kernel. It is constant in time, so it is factorised once before the time loop; each time step costs only the two sweeps. No library linear solve is used (Sec. 3.5).
- **Vectorised operations:** All stencil computations are performed using MATLAB's array operations, avoiding explicit loops over grid cells for efficiency.
- **Staggered grid:** Prevents checkerboard pressure oscillations; velocity components are stored at the faces where mass fluxes are naturally evaluated.
- **Ghost cells:** Velocity and temperature boundary conditions are implemented through ghost-cell extensions. Pressure-correction boundary conditions are imposed directly in the block coefficients of the Poisson operator, including the last-cell-centre outlet reference.

5 Analytical Solution

For fully developed flow (large z), analytical solutions exist for comparison.

5.1 Velocity Profile (Hagen–Poiseuille)

$$\frac{w}{W_{\text{in}}} = 2 [1 - (2r^*)^2] \quad (50)$$

The centreline velocity is $w_{\text{max}} = 2 W_{\text{in}}$. The hydrodynamic entry length is approximately $L_h \approx 0.05 \text{ Re } D = 5D$.

5.2 Temperature Profile

For constant wall heat flux in fully developed thermal flow:

$$\theta = 4z^* + \text{RePr} \left[\frac{1}{2}(2r^*)^2 - \frac{1}{8}(2r^*)^4 - \frac{7}{48} \right] \quad (51)$$

This form follows from direct integration of the non-dimensional energy equation (18) in the fully developed regime (Section 5.4), subject to the wall condition $\lambda \partial T / \partial r|_{\text{wall}} = q_w$ and the global energy balance $\theta_{\text{bulk}}(z^*) = 4z^*$; see Section 5.4 for the full derivation and literature sources.

The thermal entry length is $L_t \approx 0.05 \text{ Re Pr } D = 25D$.

5.3 Validity of the Fully-Developed Temperature Profile

Equation (51) makes two logically separate claims, which hold over different ranges of z^* because they rest on different physics: one about the cross-sectional *level* of the temperature (the bulk value), and one about its *radial shape*.

The bulk temperature is exact from the inlet. The bulk (mixing-cup) temperature is the flow-weighted cross-sectional average

$$\theta_{\text{bulk}}(z^*) = \frac{\int w^* \theta \, dA}{\int w^* \, dA}. \quad (52)$$

Its axial evolution follows from an energy balance on a thin slice of pipe — enthalpy gained by the stream equals heat entering through the wall:

$$\dot{m} c_v \frac{\partial T_{\text{bulk}}}{\partial z} = q_w \pi D. \quad (53)$$

Every factor on the right is constant (q_w is constant and applied from $z = 0$; the perimeter is fixed), and $\dot{m}$, c_v are constant, so $\partial T_{\text{bulk}} / \partial z$ is constant along the *entire* pipe. Non-dimensionalized this is $\partial \theta_{\text{bulk}} / \partial z^* = 4$, and integrating from $\theta_{\text{bulk}}(0) = 0$ gives

$$\theta_{\text{bulk}}(z^*) = 4z^* \quad \text{exactly, for all } z^* \geq 0, \quad (54)$$

developed or not. This balance involves only totals (total wall heat input, total enthalpy flux); it never refers to the shape of the radial profile, so the thermal entrance region does not perturb it. The one effect that would — axial conduction near the inlet — is neglected here, as specified.

The radial shape requires thermally developed flow. Reducing the energy equation (18) to the closed r^* -form in (51) assumes the profile is *thermally fully developed*: self-similar in shape, merely translating downstream, so $\partial(\theta - \theta_{\text{bulk}})/\partial z^* = 0$. That is exactly the step that makes $\partial\theta/\partial z^*$ the r^* -independent constant 4 and collapses the PDE to an ODE in r^* . Near the inlet it is false: at $z^* = 0$ the temperature is uniform ($\theta = 0$, flat); the developed shape has the wall hotter than the centreline by $\frac{11}{48}\text{RePr}$; and over the thermal entry length $L_t \approx 25D$ the profile morphs continuously between the two, a thin heated layer growing inward from the wall while the core stays at inlet temperature. Through this region $\partial(\theta - \theta_{\text{bulk}})/\partial z^*$ is large and r^* -dependent, and the ODE solution does not apply. Evaluated there anyway, (51) gives at the centreline $4z^* + \text{RePr}(-\frac{7}{48}) \approx 4z^* - 73$, i.e. negative for $z^* \lesssim 18$ — claiming the core is colder than the inlet. The numerical solution instead holds the core near $\theta = 0$ until the thermal layer reaches it ($z^* \approx 25$), then bends toward the analytical curve, the two nearly coinciding by $z^* = 50$ (Fig. 11). By contrast the hydrodynamic entry length is only $L_h \approx 0.05 \text{Re} D \approx 5D$: velocity develops about five times sooner, since here momentum diffuses $\text{Pr} = 5$ times faster than heat.

Why the level can be right while the shape is not. The constant $-\frac{7}{48}$ is fixed by requiring the flow-weighted average of the radial bracket to vanish, so (51) is *built* to satisfy $\theta_{\text{bulk}} = 4z^*$; its mean is pinned to the correct value even where its distribution about that mean is wrong. This mirrors the Hagen–Poiseuille profile (50), which always carries the correct flow rate (mean $w^* = 1$ by construction) although the true profile near the inlet is plug-like, not parabolic — right integral, wrong shape. The fully-developed profile is therefore used only as the $z^* \rightarrow \infty$ reference it is: its departure from the numerical solution at small z^* is expected behaviour, not a defect, and resolving the entrance region is the numerical solution’s role.

5.4 Nusselt Number

For hydrodynamically and thermally fully developed laminar flow in a circular pipe with *constant wall heat flux*, the Nusselt number is the exact rational value

$$\text{Nu} = \frac{hD}{\lambda} = \frac{q_w D}{\lambda(T_{\text{wall}} - T_{\text{bulk}})} = \frac{\text{RePr}}{\theta_{\text{wall}} - \theta_{\text{bulk}}} = \frac{48}{11} \approx 4.3636, \quad (55)$$

a standard textbook result for this thermal boundary condition [1, 2].

Derivation. In the fully developed region $u^* = 0$ and the profile translates rigidly in z^* , so the non-dimensional energy equation (18) reduces to

$$w^*(r^*) \frac{\partial \theta}{\partial z^*} = \frac{1}{\text{RePr}} \frac{1}{r^*} \frac{\partial}{\partial r^*} \left(r^* \frac{\partial \theta}{\partial r^*} \right). \quad (56)$$

A global energy balance over a pipe slice fixes the axial gradient: with the wall adding heat at non-dimensional rate $\partial\theta/\partial r^*|_{\text{wall}} = \text{RePr}$ and the flow carrying mean velocity $\langle w^* \rangle = 1$, the bulk (mixing-cup) temperature satisfies $\theta_{\text{bulk}}(z^*) = 4z^*$ exactly, hence $\partial\theta/\partial z^* = 4$ everywhere in the cross-section. Substituting this and $w^* = 2[1 - (2r^*)^2]$ into (56) and integrating twice, with $\partial\theta/\partial r^*|_{r^*=0} = 0$ and $\partial\theta/\partial r^*|_{r^*=1/2} = \text{RePr}$,

$$\theta(r^*, z^*) = 4z^* + \text{RePr} \left[\frac{1}{2}(2r^*)^2 - \frac{1}{8}(2r^*)^4 + c \right], \quad (57)$$

and the constant c follows from requiring the flow-weighted average of the bracket to vanish (so that $\theta_{\text{bulk}} = 4z^*$), which gives $c = -\frac{7}{48}$ — recovering Eq. (51). Evaluating the bracket at the wall gives $\theta_{\text{wall}} - \theta_{\text{bulk}} = \frac{11}{48} \text{RePr}$, and (55) follows.

Consistency check. The value is exact: (56) with its boundary conditions is a linear problem with no free parameters, so Nu is independent of Re , Pr , q_w , the fluid properties and the grid. Taking instead the assignment sheet’s literal radial coefficients $(2r^*)^2 - \frac{1}{4}(2r^*)^4 - \frac{3}{4}$ yields $\theta_{\text{wall}} - \theta_{\text{bulk}} = \frac{11}{24} \text{RePr}$ and hence $\text{Nu} = 24/11 \approx 2.18$ — exactly half of $48/11$, and inconsistent both with this classical result and with the assignment’s own stated wall condition (that literal profile has wall gradient 2RePr , and its bulk temperature is not $4z^*$).

Empirical check against the CFD solution. The solver imposes the physically consistent wall condition $\partial\theta/\partial r^* = \text{RePr}$. From the converged field ($t^* = 139.728$, $R_\theta < 10^{-6}$) the flow-weighted bulk temperature is evaluated at three downstream stations as

$$\theta_{\text{bulk}} = \frac{\sum_i w_i \theta_i r_{c,i}}{\sum_i w_i r_{c,i}}, \quad (58)$$

with w_i interpolated to the cell centres. The wall temperature is taken at the wall face, consistent with the ghost-cell boundary condition, $\theta_{\text{wall}} = \theta_{n_r} + \frac{1}{2}\Delta r \text{RePr}$, and $\text{Nu} = \text{RePr}/(\theta_{\text{wall}} - \theta_{\text{bulk}})$. The analytical fully developed values are $\theta_{\text{wall}} - \theta_{\text{bulk}} = \frac{11}{48} \text{RePr} = 114.58$ and $\text{Nu} = 48/11 = 4.364$.

z^*	θ_{bulk} (CFD)	$4z^*$ (expected)	$\theta_{\text{wall}} - \theta_{\text{bulk}}$	Nu (CFD)	deviation from $48/11$
29.9	119.59	119.6	112.22	4.455	+2.1%
39.9	159.59	159.6	113.72	4.397	+0.8%
48.9	195.59	195.6	114.22	4.378	+0.3%

The global energy balance $\theta_{\text{bulk}} = 4z^*$ is satisfied to 0.01% at all three stations, as required for a converged steady field. The CFD Nusselt number decreases monotonically along the pipe and approaches $48/11$ from above, reaching +0.3% at $z^* = 48.9$. This is the expected thermal entrance-region behaviour: in the region shortly after the thermal entry length $L_t \approx 25D$ the heated layer near the wall is still thinner than in the fully developed state, so the wall-to-bulk temperature difference is smaller and Nu is higher. (Evaluating θ_{wall} at the last cell centre instead of the wall face would bias $\theta_{\text{wall}} - \theta_{\text{bulk}}$ low by $\frac{1}{2}\Delta r \text{RePr} = 2.5$ and give $\text{Nu} = 4.48$ at $z^* = 48.9$. An earlier, unconverged snapshot at $t^* = 60$ gave θ_{bulk} up to 7% below $4z^*$ near the outlet, which shows that the full thermal convergence, beyond the radial diffusion timescale $\sim \text{RePr}R^{*2} \approx 125$, is needed for this comparison.) This is a model-free confirmation, built only from the governing PDE and the wall BC, that the physical answer is $48/11$ and not the 2.18 implied by the assignment-literal coefficients.

Why the solver can never reproduce $24/11$. The energy equation, the wall BC, and the inlet condition $\theta_{\text{in}} = 0$ are all *linear* in θ , and Nu in (55) is normalised by the wall flux. Multiplying the imposed wall flux by any constant k simply scales the entire θ field by k , leaving Nu unchanged at $48/11$. A CFD run therefore cannot reproduce $24/11$: that number exists only as a property of the internally inconsistent literal formula, never of a correctly posed simulation [1, 2, 3, 4].

6 Results

6.1 Simulation Parameters

Table 4: Simulation parameters.

Parameter	Value	Description
Re	100	Reynolds number
Pr	5	Prandtl number
$n_r \times n_z$	50×250	Grid resolution
Δr	0.01	Radial grid spacing
Δz	0.2	Axial grid spacing
Δt	0.002	Time step
n_{steps}	30 000	Maximum time steps
α_p	0.7	Pressure under-relaxation
Convective scheme	2nd-order upwind (LUD)	Stability for $Pe \gg 2$; formally 2nd-order on smooth
Time integration	Explicit Adams–Bashforth 2	2nd-order; Euler startup

6.2 Convergence History

The simulation targets convergence when the continuity, velocity, and temperature residuals fall below 10^{-6} :

$$R_{\text{cont}} = \max_{i,j} |\nabla \cdot \mathbf{u}|, \quad R_{\text{vel}} = \frac{1}{\Delta t} \max(\max |u^{n+1} - u^n|, \max |w^{n+1} - w^n|), \quad R_\theta = \frac{\max |\theta^{n+1} - \theta^n|}{\Delta t} \quad (59)$$

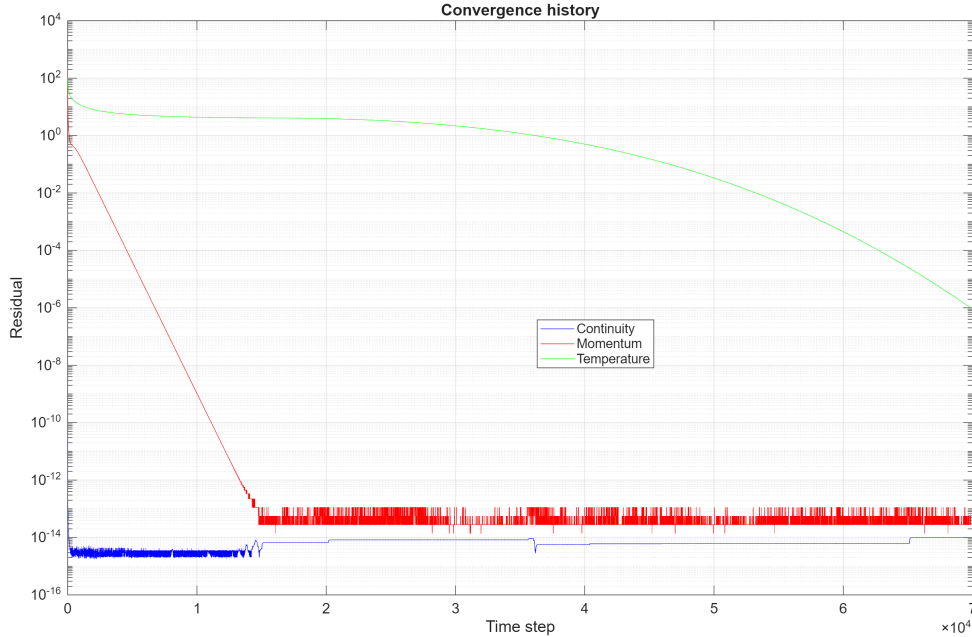


Figure 6: Convergence history: residuals versus time step.

The continuity residual (blue) is flat from the very first step rather than decaying over time: the incremental pressure correction re-solves the Poisson equation to machine

precision every single step, so $\nabla \cdot \mathbf{u}$ is driven to its numerical floor ($\sim 10^{-14}$ – 10^{-15}) at each iteration by construction, independent of whether the flow itself has reached steady state. R_{vel} (red), in contrast, measures how much the velocity field is still changing between steps: it decays exponentially, by roughly one decade every 1000 steps ($\Delta t^* = 2$), until it reaches the round-off floor at step $\approx 15,000$ ($t^* \approx 30$) – only then does it flatten into a noisy plateau rather than continuing smoothly to zero. That plateau is round-off noise: R_{vel} divides an already near-machine-precision difference $u^{n+1} - u^n$ by $\Delta t = 0.002$, amplifying floating-point noise by a factor $1/\Delta t = 500$, which is why it appears jagged at $\mathcal{O}(10^{-13})$ while R_{cont} (not divided by Δt) stays visually flat near $\mathcal{O}(10^{-14})$ – $\mathcal{O}(10^{-15})$.

In the extended MATLAB run, $R_{\text{cont}} = 9.90 \times 10^{-15}$ and $R_{\text{vel}} = 2.78 \times 10^{-14}$ at the converged time $t^* = 139.728$. The temperature residual decreases from approximately 101 at the first step and reaches $R_\theta = 9.99 \times 10^{-7}$ at step 69,864, just below the target 10^{-6} . This residual is the maximum local rate of change, not the error relative to the analytical temperature profile. Its earlier non-monotone decrease was therefore not a numerical failure: as the heated layer moved downstream and diffused inward, the cell containing the largest change moved as well. The dominant timescale is thermal diffusion across the radius, $\text{RePr}R^{*2} \approx 125$, which explains why the temperature required substantially longer to settle than the velocity. The final result is therefore a converged finite-volume solution at the stated residual tolerance, while the analytical fully-developed temperature profile remains a separate asymptotic reference for radial shape.

6.3 Velocity Field

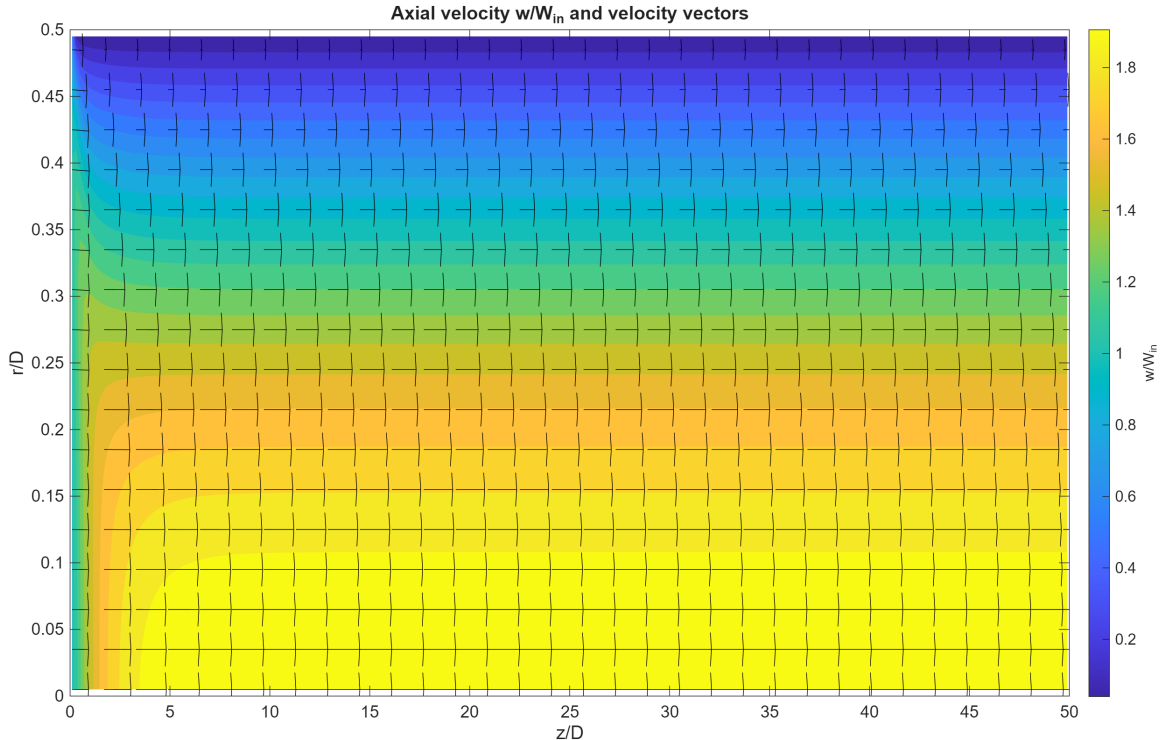


Figure 7: Axial velocity contour w/W_{in} with velocity vectors throughout the domain. The flow transitions from a uniform inlet profile ($w = 1$) to a parabolic profile ($w_{\text{max}} = 2$ at centreline); the arrows show the axial and radial components of the velocity field at regularly spaced cell centres.

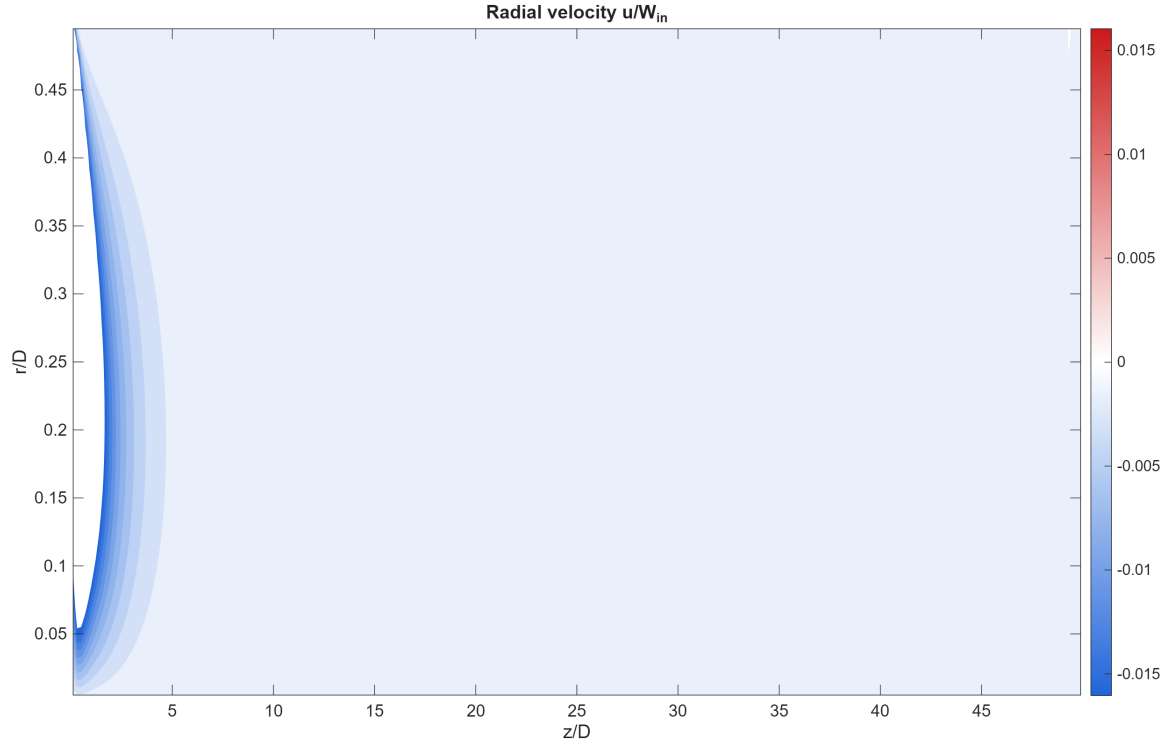


Figure 8: Radial velocity contour u/W_{in} . Non-zero only in the entrance region where the flow is developing; vanishes as the flow becomes fully developed.

6.4 Temperature Field

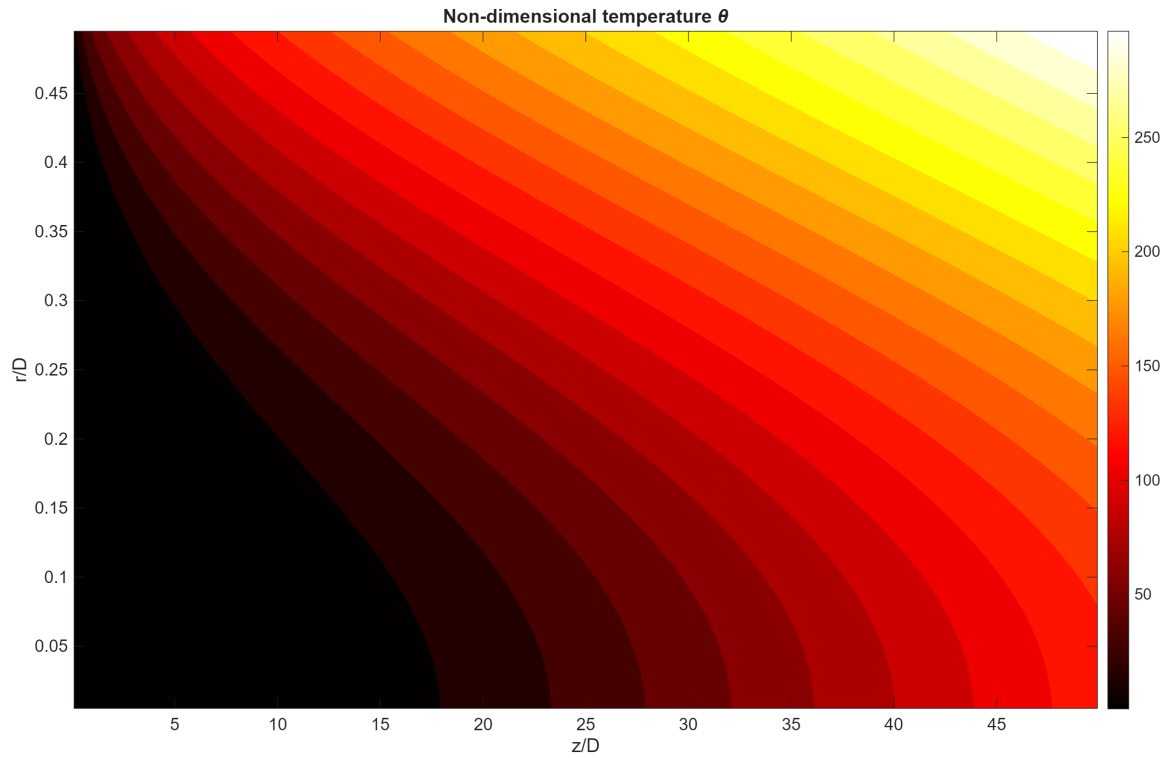


Figure 9: Non-dimensional temperature contour θ . Temperature increases along the pipe due to the constant wall heat flux.

6.5 Velocity Profiles at Selected Axial Positions

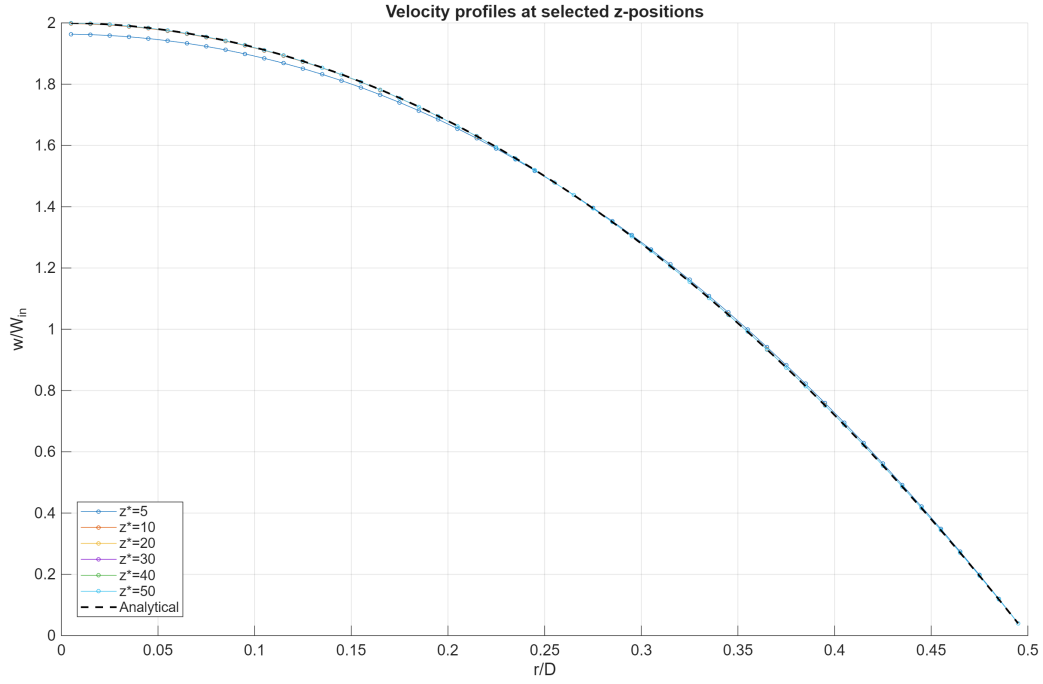


Figure 10: Axial velocity profiles $w(r)$ at $z^* = 5, 10, 20, 30, 40, 50$. Dashed lines show the analytical Hagen–Poiseuille profile. The velocity profile develops from uniform at the inlet to parabolic downstream.

6.6 Temperature Profiles at Selected Axial Positions

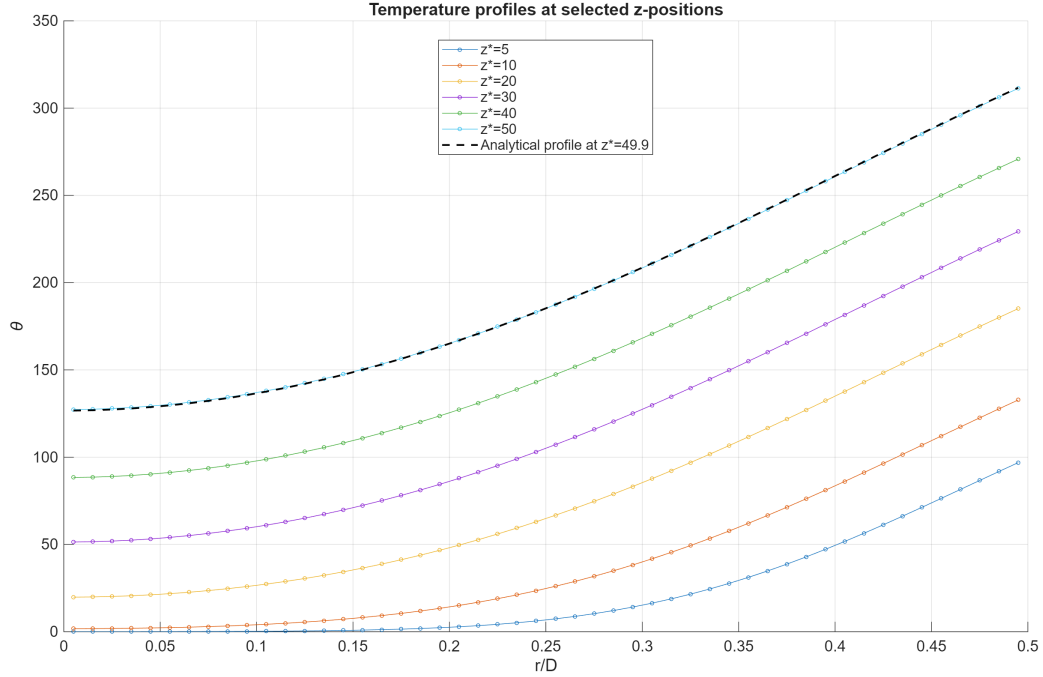


Figure 11: Non-dimensional temperature profiles $\theta(r)$ at selected axial positions (solid, numerical). The dashed curve is the analytical fully-developed profile (51), overlaid only at the largest scalar location ($z^* = 49.9$). The numerical curve has the expected developed radial shape; any remaining difference from the dashed curve reflects comparison with a fully-developed asymptotic radial profile and the finite axial location. The dashed curve is a *fully-developed asymptote* (Section 5.3): only meant to be reached for increasing z^* , not a valid prediction at every z^* — evaluated at small z^* (e.g. $z^* = 4.9$) it would extrapolate to strongly negative θ near the centreline (the $4z^*$ term not yet compensating for the formula’s negative radial offset). The final numerical field is essentially non-negative; a small, localized transient LUD undershoot occurs near the inlet–wall corner during the first time steps. Hence the asymptotic curve is not plotted at those smaller- z^* positions.

6.7 Comparison with Analytical Solution

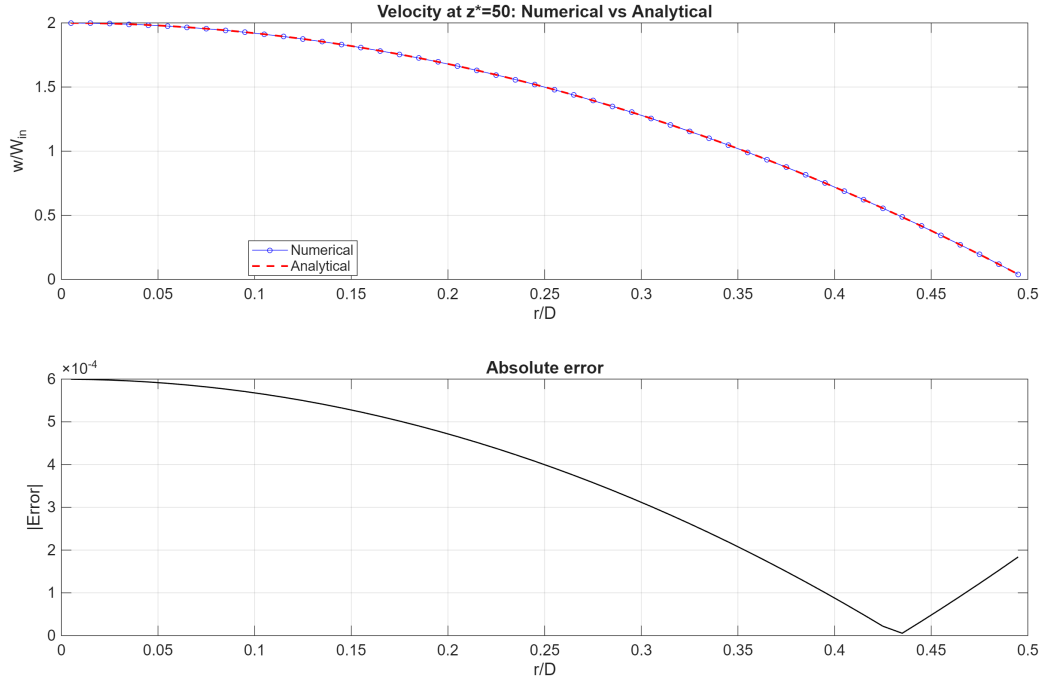


Figure 12: Velocity profile at the last cell-centre location, $z^* = 49.9$ (nominally the $z^* = 50$ outlet, fully developed): numerical vs. analytical Hagen–Poiseuille solution. The absolute-error minimum near $r^* \approx 0.43$ occurs where the signed difference between the numerical and analytical profiles changes sign. Conservation fixes the numerical area-mean velocity through the imposed inlet flow rate, which agrees with the continuous Hagen–Poiseuille mean; consequently, a small negative core error is compensated by a small positive near-wall error. The discrete cell-centre means differ slightly because of radial quadrature. The subsequent rise toward the wall is associated with local wall and cylindrical-curvature discretization, wall ghost-cell treatment, and profile sampling, not with a physical velocity minimum.

7 Conclusion

A finite volume solver for 2D axisymmetric incompressible pipe flow was developed in MATLAB and validated against analytical solutions. The solver uses a staggered grid with 2nd-order upwind (linear upwind) differencing for convective stability, explicit second-order Adams–Bashforth time integration, and an incremental projection method with one sparse direct Poisson solve per time step.

The numerical results reproduce the expected physics:

- The velocity field develops from a uniform inlet profile to the parabolic Hagen–Poiseuille profile within the hydrodynamic entry length. At $z^* = 50$ the numerical and analytical Hagen–Poiseuille profiles agree to within $\mathcal{O}(10^{-4})$ (Fig. 12) – the velocity field is essentially fully converged and validated.
- The temperature field shows the characteristic linear rise of the bulk temperature along the pipe axis ($\theta_{\text{bulk}} = 4z^*$, confirmed to within 0.01% at $z^* = 29.9, 39.9$ and 48.9; see Section 5.4) and a radial profile that approaches the analytical fully-developed solution as z^* increases (Fig. 11), as expected since that solution is only valid asymptotically. The local Nusselt number decreases along the pipe towards 48/11 and is within 0.3% of it at $z^* = 48.9$; at smaller z^* the difference is primarily the thermal entrance-region shape, while the final field satisfies the temporal convergence criterion.

A 2D Incompressible Flow in Cylindrical Coordinates

(r, z) — Velocities (u, w)

Note on notation: this appendix derives the FVM discretisation, projection method, and ghost-cell treatment in general form, illustrating u and w as cell-centred quantities near each boundary for schematic clarity. The staggered grid actually implemented (Table 2 in the main text) instead stores u exactly *at* the axis and wall faces and w exactly *at* the inlet and outlet faces, so those boundary-normal components are set directly (Dirichlet) rather than via a mirrored ghost cell; only the wall-tangential component and the scalars (p, θ) use the mirrored ghost-cell form shown here. The relaxation factor ζ below is the same quantity as α_p in Section 3.4, where this submission uses $\alpha_p = 0.7$.

A.1 Governing Equations (Strong Form)

For axisymmetric flow ($\partial/\partial\phi = 0$, $v_\phi = 0$) in cylindrical coordinates (r, z) :

Continuity:

$$\frac{1}{r} \frac{\partial(ru)}{\partial r} + \frac{\partial w}{\partial z} = 0 \quad (60)$$

Radial momentum (r -direction):

$$\frac{\partial u}{\partial t} + \frac{1}{r} \frac{\partial(ru^2)}{\partial r} + \frac{\partial(uw)}{\partial z} = -\frac{1}{\rho} \frac{\partial p}{\partial r} + \frac{1}{\rho r} \frac{\partial(r\tau_{rr})}{\partial r} + \frac{1}{\rho} \frac{\partial\tau_{rz}}{\partial z} \quad (61)$$

Axial momentum (z -direction):

$$\frac{\partial w}{\partial t} + \frac{1}{r} \frac{\partial(ruw)}{\partial r} + \frac{\partial(w^2)}{\partial z} = -\frac{1}{\rho} \frac{\partial p}{\partial z} + \frac{1}{\rho r} \frac{\partial(r\tau_{rz})}{\partial r} + \frac{1}{\rho} \frac{\partial\tau_{zz}}{\partial z} \quad (62)$$

where for a Newtonian fluid with constant viscosity $\mu = \rho\nu$:

$$\tau_{rr} = 2\mu \frac{\partial u}{\partial r} \quad (63)$$

$$\tau_{zz} = 2\mu \frac{\partial w}{\partial z} \quad (64)$$

$$\tau_{rz} = \mu \left(\frac{\partial u}{\partial z} + \frac{\partial w}{\partial r} \right) \quad (65)$$

A.2 Staggered Grid Layout

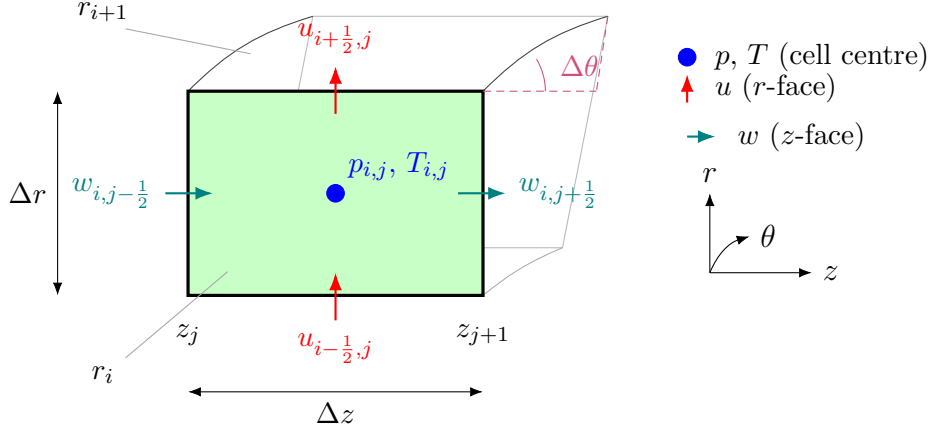


Figure 13: Staggered grid in the (r, z) plane. Scalars (p, T) at the cell centre (blue); radial velocity u on the (curved, cylindrical) r -faces (red); axial velocity w on the z -faces (teal). The control volume is drawn swept through a finite $\Delta\theta$ purely to convey the 3-D cylindrical shape of the r -faces (note the outer face at r_{i+1} sweeps further than the inner face at r_i , since arc length = $r \Delta\theta$); the axisymmetric problem itself has $\partial/\partial\theta = 0$ and no θ -dependence.

A.3 Finite Volume Discretisation — Integration over $\Delta r \Delta z$

The volume element for an axisymmetric ring CV is $dV = r dr d\phi dz$. After integrating over $\phi \in [0, 2\pi]$, every integral acquires the factor 2π . Since the same factor appears on both sides of the momentum equations, it cancels and we work with the *area* element $r dr dz$.

A.3.1 Continuity Equation

Integrate Eq. (60) over the scalar CV $[r_{i-\frac{1}{2}}, r_{i+\frac{1}{2}}] \times [z_{j-\frac{1}{2}}, z_{j+\frac{1}{2}}]$:

$$\int_{r_{i-\frac{1}{2}}}^{r_{i+\frac{1}{2}}} \int_{z_{j-\frac{1}{2}}}^{z_{j+\frac{1}{2}}} \left[\frac{1}{r} \frac{\partial(ru)}{\partial r} + \frac{\partial w}{\partial z} \right] r dr dz = 0 \quad (66)$$

Applying the FV divergence theorem:

$$\boxed{\frac{r_{i+\frac{1}{2}} u_{i+\frac{1}{2},j} - r_{i-\frac{1}{2}} u_{i-\frac{1}{2},j}}{r_i \Delta r} + \frac{w_{i,j+\frac{1}{2}} - w_{i,j-\frac{1}{2}}}{\Delta z} = 0} \quad (67)$$

A.3.2 Axial Momentum: CV at $(i, j + \frac{1}{2})$

Integrate Eq. (62) over $[r_{i-\frac{1}{2}}, r_{i+\frac{1}{2}}] \times [z_j, z_{j+1}]$, weighting by r :

Temporal term:

$$\frac{\partial w_{i,j+\frac{1}{2}}}{\partial t} \approx \frac{w_{i,j+\frac{1}{2}}^{n+1} - w_{i,j+\frac{1}{2}}^n}{\Delta t} \quad (68)$$

Convective terms:

$$u \frac{\partial w}{\partial r} \Big|_{i,j+\frac{1}{2}} \approx \bar{u}_{i,j+\frac{1}{2}} \frac{w_{i+1,j+\frac{1}{2}} - w_{i-1,j+\frac{1}{2}}}{2\Delta r} \quad (69)$$

$$w \frac{\partial w}{\partial z} \Big|_{i,j+\frac{1}{2}} \approx w_{i,j+\frac{1}{2}} \frac{w_{i,j+\frac{3}{2}} - w_{i,j-\frac{1}{2}}}{2\Delta z} \quad (70)$$

where $\bar{u}_{i,j+\frac{1}{2}} = \frac{1}{4}(u_{i+\frac{1}{2},j} + u_{i-\frac{1}{2},j} + u_{i+\frac{1}{2},j+1} + u_{i-\frac{1}{2},j+1})$.

Pressure gradient:

$$\frac{1}{\rho} \frac{\partial p}{\partial z} \Big|_{i,j+\frac{1}{2}} \approx \frac{1}{\rho} \frac{p_{i,j+1} - p_{i,j}}{\Delta z} \quad (71)$$

Viscous diffusion (using $\frac{1}{r} \frac{\partial}{\partial r}(r \frac{\partial w}{\partial r})$):

$$\nu \left(\frac{\partial^2 w}{\partial r^2} + \frac{1}{r} \frac{\partial w}{\partial r} \right) \approx \frac{\nu}{r_i \Delta r} \left(r_{i+\frac{1}{2}} \frac{w_{i+1,j+\frac{1}{2}} - w_{i,j+\frac{1}{2}}}{\Delta r} - r_{i-\frac{1}{2}} \frac{w_{i,j+\frac{1}{2}} - w_{i-1,j+\frac{1}{2}}}{\Delta r} \right) \quad (72)$$

$$\nu \frac{\partial^2 w}{\partial z^2} \approx \nu \frac{w_{i,j+\frac{3}{2}} - 2w_{i,j+\frac{1}{2}} + w_{i,j-\frac{1}{2}}}{\Delta z^2} \quad (73)$$

A.3.3 Radial Momentum: CV at $(i + \frac{1}{2}, j)$

Integrate Eq. (61) over $[r_i, r_{i+1}] \times [z_{j-\frac{1}{2}}, z_{j+\frac{1}{2}}]$:

Convective terms:

$$u \frac{\partial u}{\partial r} \Big|_{i+\frac{1}{2},j} \approx u_{i+\frac{1}{2},j} \frac{u_{i+\frac{3}{2},j} - u_{i-\frac{1}{2},j}}{2\Delta r} \quad (74)$$

$$w \frac{\partial u}{\partial z} \Big|_{i+\frac{1}{2},j} \approx \bar{w}_{i+\frac{1}{2},j} \frac{u_{i+\frac{1}{2},j+1} - u_{i+\frac{1}{2},j-1}}{2\Delta z} \quad (75)$$

Pressure gradient:

$$\frac{1}{\rho} \frac{\partial p}{\partial r} \Big|_{i+\frac{1}{2},j} \approx \frac{1}{\rho} \frac{p_{i+1,j} - p_{i,j}}{\Delta r} \quad (76)$$

Viscous diffusion:

$$\nu \left(\frac{\partial^2 u}{\partial r^2} + \frac{1}{r} \frac{\partial u}{\partial r} - \frac{u}{r^2} \right) \approx \frac{\nu}{r_{i+\frac{1}{2}} \Delta r} \left(r_{i+1} \frac{u_{i+\frac{3}{2},j} - u_{i+\frac{1}{2},j}}{\Delta r} - r_i \frac{u_{i+\frac{1}{2},j} - u_{i-\frac{1}{2},j}}{\Delta r} \right) - \frac{\nu u_{i+\frac{1}{2},j}}{r_{i+\frac{1}{2}}^2} \quad (77)$$

$$\nu \frac{\partial^2 u}{\partial z^2} \approx \nu \frac{u_{i+\frac{1}{2},j+1} - 2u_{i+\frac{1}{2},j} + u_{i+\frac{1}{2},j-1}}{\Delta z^2} \quad (78)$$

A.4 Projection Method

A.4.1 Step 1 — Predictor

Advance without the pressure gradient (explicit Euler or AB2):

$$\boxed{w_{i,j+\frac{1}{2}}^* = w_{i,j+\frac{1}{2}}^n + \Delta t \left[-\bar{u}^n \frac{w_{i+1,j+\frac{1}{2}}^n - w_{i-1,j+\frac{1}{2}}^n}{2\Delta r} - w_{i,j+\frac{1}{2}}^n \frac{w_{i,j+\frac{3}{2}}^n - w_{i,j-\frac{1}{2}}^n}{2\Delta z} + \nu D_r^w + \nu D_z^w \right]} \quad (79)$$

$$u_{i+\frac{1}{2},j}^* = u_{i+\frac{1}{2},j}^n + \Delta t \left[-u_{i+\frac{1}{2},j}^n \frac{u_{i+\frac{3}{2},j}^n - u_{i-\frac{1}{2},j}^n}{2\Delta r} - \bar{w}^n \frac{u_{i+\frac{1}{2},j+1}^n - u_{i+\frac{1}{2},j-1}^n}{2\Delta z} + \nu D_r^u + \nu D_z^u \right] \quad (80)$$

where the diffusion operators are:

$$D_r^w = \frac{1}{r_i \Delta r} \left(r_{i+\frac{1}{2}} \frac{w_{i+1,j+\frac{1}{2}}^n - w_{i,j+\frac{1}{2}}^n}{\Delta r} - r_{i-\frac{1}{2}} \frac{w_{i,j+\frac{1}{2}}^n - w_{i-1,j+\frac{1}{2}}^n}{\Delta r} \right), \quad D_z^w = \frac{w_{i,j+\frac{3}{2}}^n - 2w_{i,j+\frac{1}{2}}^n + w_{i,j-\frac{1}{2}}^n}{\Delta z^2} \quad (81)$$

$$D_r^u = \frac{r_{i+1}(u_{i+\frac{3}{2},j}^n - u_{i+\frac{1}{2},j}^n) - r_i(u_{i+\frac{1}{2},j}^n - u_{i-\frac{1}{2},j}^n)}{r_{i+\frac{1}{2}} \Delta r^2} - \frac{u_{i+\frac{1}{2},j}^n}{r_{i+\frac{1}{2}}^2} \quad (82)$$

$$D_z^u = \frac{u_{i+\frac{1}{2},j+1}^n - 2u_{i+\frac{1}{2},j}^n + u_{i+\frac{1}{2},j-1}^n}{\Delta z^2} \quad (83)$$

A.4.2 Step 2 — Poisson Equation for Pressure Correction p'

Enforcing the discrete continuity Eq. (67) after correction:

$$\frac{1}{r} \frac{\partial}{\partial r} \left(r \frac{\partial p'}{\partial r} \right) + \frac{\partial^2 p'}{\partial z^2} = \frac{\rho}{\Delta t} \left(\frac{1}{r} \frac{\partial (ru^*)}{\partial r} + \frac{\partial w^*}{\partial z} \right) \quad (84)$$

Discretised form at cell (i, j) :

$$\frac{r_{i+\frac{1}{2}}(p'_{i+1,j} - p'_{i,j}) - r_{i-\frac{1}{2}}(p'_{i,j} - p'_{i-1,j})}{r_i \Delta r^2} + \frac{p'_{i,j+1} - 2p'_{i,j} + p'_{i,j-1}}{\Delta z^2} = \frac{\rho}{\Delta t} \left(\frac{r_{i+\frac{1}{2}} u_{i+\frac{1}{2},j}^* - r_{i-\frac{1}{2}} u_{i-\frac{1}{2},j}^*}{r_i \Delta r} + \frac{w_{i,j+\frac{1}{2}}^* - w_{i,j-\frac{1}{2}}^*}{\Delta z} \right) \quad (85)$$

A.4.3 Step 3 — Velocity Correction

$$w_{i,j+\frac{1}{2}}^{n+1} = w_{i,j+\frac{1}{2}}^* - \frac{\Delta t}{\rho} \frac{p'_{i,j+1} - p'_{i,j}}{\Delta z} \quad (86)$$

$$u_{i+\frac{1}{2},j}^{n+1} = u_{i+\frac{1}{2},j}^* - \frac{\Delta t}{\rho} \frac{p'_{i+1,j} - p'_{i,j}}{\Delta r} \quad (87)$$

A.4.4 Step 4 — Pressure Update

$$p_{i,j}^{n+1} = p_{i,j}^n + \zeta p'_{i,j}, \quad \zeta = \alpha_p = 0.7 \text{ (this submission)} \quad (88)$$

A.5 Poisson Solver Procedure

Rearranging Eq. (85) to isolate $p'_{i,j}$:

$$p'_{i,j} = \frac{b_{i,j} - \frac{r_{i+\frac{1}{2}} p'_{i+1,j} + r_{i-\frac{1}{2}} p'_{i-1,j}}{r_i \Delta r^2} - \frac{p'_{i,j+1} + p'_{i,j-1}}{\Delta z^2}}{-\frac{r_{i+\frac{1}{2}} + r_{i-\frac{1}{2}}}{r_i \Delta r^2} - \frac{2}{\Delta z^2}} \quad (89)$$

Gauss–Seidel / SOR iteration:

1. Set $p' = 0$ everywhere.
2. Enforce BCs: $\partial p'/\partial r = 0$ at wall & centreline; $\partial p'/\partial z = 0$ at inlet/outlet (or $p' = 0$ at outlet).
3. Sweep interior cells with Eq. (89); apply SOR: $p'^{\text{new}} = p'^{\text{old}} + \omega(p'^{\text{GS}} - p'^{\text{old}})$.
4. Converge when $\max |p'^{\text{new}} - p'^{\text{old}}| < \varepsilon$.

Direct solve (used in this work): group the unknowns by axial station, so that the operator becomes block tridiagonal, and eliminate it with a block Thomas algorithm built on a hand-written LU kernel. The assignment asks for a direct method that is programmed, so no library solve (`spsolve`, MATLAB `A\b`) is used; see Sec. 3.5 of the main report. Note: fix one pressure value (e.g. $p'_{1,N_z} = 0$ at the outlet corner) to remove the null space.

A.6 Ghost Cells and Boundary Conditions

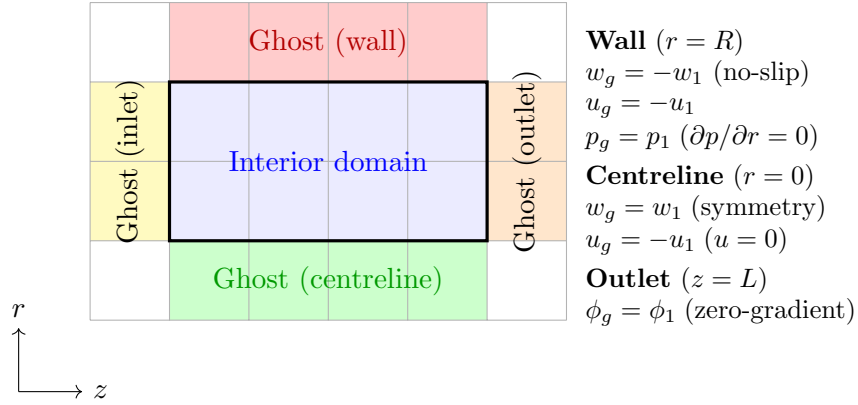


Figure 14: Ghost cell layout for the (r, z) domain. Green: centreline symmetry; red: no-slip wall; yellow: inlet; orange: outlet. Subscript g = ghost, subscript 1 = first interior cell.

Boundary	Condition	Variable	Ghost value
Centreline ($r = 0$)	$u = 0$	u	$u_g = -u_1$
	$\partial w/\partial r = 0$	w	$w_g = w_1$
	$\partial p/\partial r = 0$	p	$p_g = p_1$
Wall ($r = R$)	$u = 0, w = 0$	u	$u_g = -u_1$
		w	$w_g = -w_1$
	$\partial p/\partial r = 0$	p	$p_g = p_1$
	$\lambda \partial T/\partial r = q_w$	T	$T_g = T_1 + \frac{q_w \Delta r}{\lambda}$
Inlet ($z = 0$)	$u = 0, w = W_{\text{in}}$	w	$w_g = 2W_{\text{in}} - w_1$
		u	$u_g = -u_1$
	$T = T_{\text{in}}$	T	$T_g = 2T_{\text{in}} - T_1$
Outlet ($z = L$)	$\partial(\cdot)/\partial z = 0$	any	$\phi_g = \phi_1$

B Temperature Formula and Nusselt Number Audit

This appendix derives from first principles why the fully developed Nusselt number for the present problem is exactly $\text{Nu} = 48/11 \approx 4.3636$, and shows which Nusselt number each of the two candidate fully developed temperature profiles implies: the one printed on the assignment sheet and the corrected one used throughout this report and the code.

All quantities are non-dimensional in the scaling of the assignment:

$$r^* = \frac{r}{D}, \quad z^* = \frac{z}{D}, \quad w^* = \frac{w}{W_{\text{in}}}, \quad \theta = \frac{(T - T_{\text{in}})\rho c_v W_{\text{in}}}{q_w}, \quad \text{Re} = \frac{\rho W_{\text{in}} D}{\mu} = 100, \quad \text{Pr} = \frac{\mu c_v}{\lambda} = 5. \quad (90)$$

It is convenient to introduce

$$s \equiv 2r^* = \frac{2r}{D} \in [0, 1], \quad (91)$$

so that the centreline is at $s = 0$ and the pipe wall at $s = 1$.

B.1 The Two Candidate Fully Developed Profiles

Both candidates have the form

$$\theta(r^*, z^*) = 4z^* + \text{RePr} B(s), \quad (92)$$

and differ only in the radial bracket $B(s)$:

$$B_{\text{corr}}(s) = \frac{1}{2}s^2 - \frac{1}{8}s^4 - \frac{7}{48} \quad (\text{corrected, used in report and code}), \quad (93)$$

$$B_{\text{lit}}(s) = s^2 - \frac{1}{4}s^4 - \frac{3}{4} \quad (\text{assignment-literal}). \quad (94)$$

The corrected bracket is implemented in `analytical_temperature.m` (Listing 29) and is Eq. (51) of the report.

B.2 Governing Equation and Axial Gradient

For hydrodynamically and thermally fully developed axisymmetric flow ($u^* = 0$, the *shape* of the profile no longer changes with z^*), with viscous dissipation and axial conduction neglected, the non-dimensional energy equation reduces to

$$w^*(r^*) \frac{\partial \theta}{\partial z^*} = \frac{1}{\text{RePr}} \frac{1}{r^*} \frac{\partial}{\partial r^*} \left(r^* \frac{\partial \theta}{\partial r^*} \right). \quad (95)$$

A global energy balance over a pipe slice fixes the axial gradient. The wall adds heat at the non-dimensional rate $\partial \theta / \partial r^*|_{\text{wall}} = \text{RePr}$ per unit wall area (Eq. (101)), and the enthalpy flux carried by the flow is $\int w^* \theta \, dA$. With $\langle w^* \rangle = 1$ (the parabolic profile carries the same mean velocity as the uniform inlet), the bulk (mixing-cup) temperature

$$\theta_{\text{bulk}} \equiv \frac{\int w^* \theta \, dA}{\int w^* \, dA} \quad (96)$$

satisfies

$$\frac{d\theta_{\text{bulk}}}{dz^*} = 4 \quad \implies \quad \theta_{\text{bulk}}(z^*) = 4z^* \quad (97)$$

exactly, at every z^* . This is energy conservation from the inlet and is independent of the radial profile shape. In the fully developed region the whole profile translates uniformly in z^* , so

$$\frac{\partial \theta}{\partial z^*} = \frac{d\theta_{\text{bulk}}}{dz^*} = 4 \quad (98)$$

everywhere in the cross-section.

B.3 Velocity Profile and Wall Boundary Condition

The Hagen–Poiseuille profile (Eq. (50)) is

$$w^*(r^*) = 2 [1 - (2r^*)^2] = 2(1 - s^2). \quad (99)$$

Fourier’s law at the heated wall,

$$\lambda \frac{\partial T}{\partial r} \Big|_{\text{wall}} = q_w, \quad (100)$$

non-dimensionalises directly:

$$\frac{\partial \theta}{\partial r^*} \Big|_{\text{wall}} = D \frac{\rho c_v W_{\text{in}}}{q_w} \frac{\partial T}{\partial r} \Big|_{\text{wall}} = D \frac{\rho c_v W_{\text{in}}}{q_w} \cdot \frac{q_w}{\lambda} = \frac{\rho c_v W_{\text{in}} D}{\lambda} = \text{RePr}. \quad (101)$$

Numerically $\text{RePr} = 500$ here, not an order-one value: the reference length and temperature scales do not cancel. The symmetry condition at the axis is

$$\frac{\partial \theta}{\partial r^*} \Big|_{r^*=0} = 0. \quad (102)$$

B.4 Integration of the Radial Equation

Inserting Eqs. (97) and (99) into Eq. (95) gives

$$\frac{1}{r^*} \frac{d}{dr^*} \left(r^* \frac{d\theta}{dr^*} \right) = \text{RePr} \cdot 4 \cdot w^*(r^*) = 8\text{RePr} (1 - 4r^{*2}). \quad (103)$$

Integrating once and applying regularity ($r^* d\theta/dr^* \rightarrow 0$ as $r^* \rightarrow 0$),

$$r^* \frac{d\theta}{dr^*} = 8\text{RePr} \left(\frac{r^{*2}}{2} - r^{*4} \right), \quad (104)$$

$$\frac{d\theta}{dr^*} = 8\text{RePr} r^* \left(\frac{1}{2} - r^{*2} \right). \quad (105)$$

At the wall, $r^* = \frac{1}{2}$,

$$\frac{d\theta}{dr^*} \Big|_{r^*=1/2} = 8\text{RePr} \cdot \frac{1}{2} \cdot \left(\frac{1}{2} - \frac{1}{4} \right) = \text{RePr}, \quad (106)$$

which matches the wall condition (101). Integrating a second time,

$$\theta(r^*, z^*) = 2\text{RePr} (r^{*2} - r^{*4}) + C(z^*) = \text{RePr} \left(\frac{1}{2} s^2 - \frac{1}{8} s^4 \right) + C(z^*), \quad s = 2r^*. \quad (107)$$

B.5 Integration Constant from the Global Energy Balance

Write $C(z^*) = 4z^* + \text{RePr} c$ with c a pure constant. The energy balance $\theta_{\text{bulk}} = 4z^*$ (Eq. (97)) requires the flow-weighted mean of the radial bracket to vanish:

$$\left\langle \frac{1}{2} s^2 - \frac{1}{8} s^4 + c \right\rangle_w = 0, \quad \langle f \rangle_w \equiv \frac{\int_0^{1/2} f w^* r^* dr^*}{\int_0^{1/2} w^* r^* dr^*}. \quad (108)$$

With $r^* = s/2$ and Eq. (99),

$$w^* r^* dr^* = \frac{1}{2} s (1 - s^2) ds, \quad \int_0^1 \frac{1}{2} s (1 - s^2) ds = \frac{1}{8}, \quad (109)$$

so that

$$\left\langle \frac{1}{2} s^2 - \frac{1}{8} s^4 \right\rangle_w = 8 \int_0^1 \left(\frac{1}{2} s^2 - \frac{1}{8} s^4 \right) \frac{1}{2} s (1 - s^2) ds = 8 \cdot \frac{7}{384} = \frac{7}{48}. \quad (110)$$

Hence $c = -7/48$, and the corrected fully developed profile is

$$\theta(r^*, z^*) = 4z^* + \text{RePr} \left[\frac{1}{2} (2r^*)^2 - \frac{1}{8} (2r^*)^4 - \frac{7}{48} \right]. \quad (111)$$

Naively halving the assignment's constant $-\frac{3}{4}$ would give $-\frac{3}{8}$, which is wrong: the constant must be re-derived from the energy balance and equals $-\frac{7}{48}$.

B.6 Nusselt Number of the Corrected Profile

The Nusselt number is $\text{Nu} = hD/\lambda$ with $h = q_w/(T_{\text{wall}} - T_{\text{bulk}})$. Non-dimensionalising the wall-to-bulk temperature difference,

$$T_{\text{wall}} - T_{\text{bulk}} = (\theta_{\text{wall}} - \theta_{\text{bulk}}) \frac{q_w}{\rho c_v W_{\text{in}}}, \quad (112)$$

gives

$$\text{Nu} = \frac{q_w D}{\lambda (T_{\text{wall}} - T_{\text{bulk}})} = \frac{\rho c_v W_{\text{in}} D / \lambda}{\theta_{\text{wall}} - \theta_{\text{bulk}}} = \frac{\text{RePr}}{\theta_{\text{wall}} - \theta_{\text{bulk}}}. \quad (113)$$

From Eq. (111), the bracket at the wall ($s = 1$) is

$$B_{\text{corr}}(1) = \frac{1}{2} - \frac{1}{8} - \frac{7}{48} = \frac{24}{48} - \frac{6}{48} - \frac{7}{48} = \frac{11}{48}, \quad (114)$$

while θ_{bulk} corresponds to a bracket value of 0. Therefore

$$\theta_{\text{wall}} - \theta_{\text{bulk}} = \frac{11}{48} \text{RePr}, \quad \text{Nu} = \frac{\text{RePr}}{(11/48) \text{RePr}} = \frac{48}{11} \approx 4.3636. \quad (115)$$

B.7 Why the Value is Exactly 48/11

Equations (95)–(101) form a linear two-point boundary-value problem with fixed coefficients and no free parameters. Given only

- hydrodynamically *and* thermally fully developed flow,
- constant wall heat flux (the H1 thermal boundary condition),
- laminar flow with the parabolic velocity profile (99),
- constant fluid properties,
- negligible axial conduction and viscous dissipation,

the radial profile (111) and the ratio (115) are completely determined. Re , Pr , q_w , D , the fluid properties and the numerical grid all cancel out of Nu : they rescale θ but not the dimensionless ratio $(\theta_{\text{wall}} - \theta_{\text{bulk}})/(\text{RePr})$. The value $48/11$ is a rational number obtained by integration, not an empirical correlation, and it is the classical result for this configuration [1, 2, 3, 4]. For contrast, a constant wall *temperature* gives the different value $\text{Nu} \approx 3.66$ (from the first eigenvalue of the Graetz problem); $48/11$ is specifically the constant-heat-flux value relevant to this assignment.

B.8 Assignment-Literal Profile: $\text{Nu} = 24/11$

Taking the printed bracket $B_{\text{lit}}(s) = s^2 - \frac{1}{4}s^4 - \frac{3}{4}$ at face value:

Wall value.

$$B_{\text{lit}}(1) = 1 - \frac{1}{4} - \frac{3}{4} = 0 \quad \implies \quad \theta_{\text{wall}} = 4z^*. \quad (116)$$

Bulk value. With the same flow-weighted average,

$$\left\langle s^2 - \frac{1}{4}s^4 \right\rangle_w = 8 \int_0^1 \left(s^2 - \frac{1}{4}s^4 \right) \frac{1}{2} s (1 - s^2) ds = 8 \cdot \frac{7}{192} = \frac{7}{24}, \quad (117)$$

$$\langle B_{\text{lit}} \rangle_w = \frac{7}{24} - \frac{3}{4} = \frac{7}{24} - \frac{18}{24} = -\frac{11}{24}. \quad (118)$$

The literal formula therefore gives

$$\theta_{\text{bulk,lit}} = 4z^* - \frac{11}{24}\text{RePr} \neq 4z^*, \quad (119)$$

i.e. it violates the global energy balance (97), a second, independent symptom of the same defect.

Nusselt number. From Eq. (113),

$$\theta_{\text{wall}} - \theta_{\text{bulk}} = 0 - \left(-\frac{11}{24}\text{RePr} \right) = \frac{11}{24}\text{RePr}, \quad (120)$$

$$\text{Nu}_{\text{lit}} = \frac{\text{RePr}}{(11/24)\text{RePr}} = \frac{24}{11} \approx 2.1818 = \frac{1}{2} \cdot \frac{48}{11}. \quad (121)$$

Origin of the factor of two. The literal radial coefficients are $(1, \frac{1}{4})$ where the integration (107) gives $(\frac{1}{2}, \frac{1}{8})$, each doubled. Equivalently, the wall slope of the literal profile is

$$\frac{d}{dr^*} \left[\text{RePr} \left(s^2 - \frac{1}{4}s^4 \right) \right]_{s=1} = \text{RePr} (4s - 2s^3) \Big|_{s=1} = 2\text{RePr}, \quad (122)$$

twice the physical wall gradient RePr required by the assignment's own stated condition $\lambda \partial T / \partial r|_{\text{wall}} = q_w$ (Eq. (101)). A wall gradient twice as steep produces a wall-to-bulk temperature difference twice as large and, since $\text{Nu} \propto 1/(\theta_{\text{wall}} - \theta_{\text{bulk}})$, a Nusselt number half as large.

B.9 Summary

With $k \equiv \text{RePr}$ ($= 500$ for $\text{Re} = 100$, $\text{Pr} = 5$), Table 5 compares the two profiles. The empirical cross-check against the converged CFD solution is given in Section 5.4.

Table 5: Consistency of the corrected and assignment-literal fully developed temperature profiles, $k = \text{RePr}$.

Quantity	Corrected profile	Assignment-literal profile
Radial bracket $B(s)$, $s = 2r^*$	$\frac{1}{2}s^2 - \frac{1}{8}s^4 - \frac{7}{48}$	$s^2 - \frac{1}{4}s^4 - \frac{3}{4}$
Wall gradient $\partial\theta/\partial r^* _{r^*=1/2}$	k	$2k$
B at the wall ($s = 1$)	$\frac{11}{48}$	0
Flow-weighted mean $\langle B \rangle_w$	0	$-\frac{11}{24}$
θ_{bulk}	$4z^*$	$4z^* - \frac{11}{24}k$
$\theta_{\text{wall}} - \theta_{\text{bulk}}$	$\frac{11}{48}k$	$\frac{11}{24}k$
Analytical Nusselt number	$48/11 \approx 4.3636$	$24/11 \approx 2.1818$
Energy balance $\theta_{\text{bulk}} = 4z^*$	Satisfied	Violated
Agreement with theory	Matches [1, 2] exactly	Half the textbook value; inconsistent with the stated wall BC
Nu from CFD (Section 5.4)	4.455 ($z^* = 29.9$) to 4.378 ($z^* = 48.9$), i.e. within 0.3% of 48/11 at the outlet	Not attainable (linear problem, flux-normalised Nu)

Thus the assignment-literal coefficients are not an alternative physical solution; they are a transcription with the radial terms doubled, equivalently missing the factor $\frac{1}{2}$ on RePr , and with $-\frac{3}{4}$ in place of the energy-balance constant $-\frac{7}{48}$. Because the temperature equation, the wall boundary condition and the inlet condition $\theta_{\text{in}} = 0$ are linear in θ , multiplying the imposed wall heat flux by any constant k' only scales the whole temperature field, $\theta \rightarrow k'\theta$. The flux-normalised Nusselt number (113) therefore remains 48/11 for the correctly posed problem, and no simulation can reproduce 24/11.

C MATLAB Source Code

The complete MATLAB source code for the AB2 solver is listed below. Each function corresponds to a module in Table 3.

C.1 Configuration and Initialisation

```
1 function c = constants()
2 % CONSTANTS Physical constants and simulation parameters.
3 % c = constants() returns a struct with all parameters.
4
5 % Non-dimensional numbers
6 c.Re = 100.0; % Reynolds number
7 c.Pr = 5.0; % Prandtl number
8
9 % Geometry (non-dimensional)
10 c.L_over_D = 50.0; % Pipe length in diameters
11
12 % Grid parameters
13 c.n_r = 50; % Radial grid cells
14 c.n_z = 250; % Axial grid cells
15
16 % Time parameters
17 c.dt = 0.002; % Time step
18 c.n_steps = 100000; % Max time steps
19 c.output_interval = 1000;
20
21 % Numerical parameters
22 c.tol_continuity = 1e-6;
23 c.tol_velocity = 1e-6;
24 c.tol_temperature = 1e-6;
25 c.alpha_p = 0.7; % Pressure under-relaxation
26 end
```

Listing 1: constants.m — Physical parameters, grid size, and solver tolerances

```
1 function [u, w, p, T] = initialise_fields(n_r, n_z)
2 % INITIALISE_FIELDS Allocate and set initial conditions.
3 %
4 % [u, w, p, T] = initialise_fields(n_r, n_z)
5 %
6 % u : (n_r+1, n_z) radial velocity at r-faces -> zero
7 % w : (n_r, n_z+1) axial velocity at z-faces -> uniform 1
8 % p : (n_r, n_z) pressure at cell centres -> zero
9 % T : (n_r, n_z) temperature at cell centres -> zero
10
11 u = zeros(n_r + 1, n_z);
12 w = ones(n_r, n_z + 1);
13 p = zeros(n_r, n_z);
14 T = zeros(n_r, n_z);
15 end
```

Listing 2: initialise_fields.m — Allocation and initial conditions for all fields

C.2 Main Simulation Script

```

1 %% MAIN Simulation loop for 2D axisymmetric incompressible pipe flow.
2 %
3 % Second-order Adams-Bashforth integration with one incremental
4 % correction per step. Forward Euler initialises the multistep history.
5
6 clear; close all; clc;
7
8 %% Load parameters
9 c = constants();
10 Re = c.Re; Pr = c.Pr;
11 n_r = c.n_r; n_z = c.n_z;
12 dt = c.dt; n_steps = c.n_steps;
13 alpha_p = c.alpha_p;
14
15 fprintf('=====\\n'
16 );
17 fprintf('Incompressible Pipe Flow Solver (MATLAB)\\n');
18 fprintf(' Re=%g, Pr=%g, n_r=%d, n_z=%d\\n', Re, Pr, n_r, n_z);
19 fprintf(' dt=%g, max steps=%d\\n', dt, n_steps);
20 fprintf('=====\\n'
21 );
22
23 %% Grid
24 [r_c, r_f, z_c, z_f, dr, dz] = make_grid(n_r, n_z, c.L_over_D);
25
26 % CFL check
27 CFL_conv = max(dt/dr, dt/dz);
28 CFL_visc = dt * (1/Re) * (1/dr^2 + 1/dz^2);
29 fprintf(' CFL_conv = %.4f (should be < 1)\\n', CFL_conv);
30 fprintf(' CFL_visc = %.4f (should be < 0.5)\\n', CFL_visc);
31 if CFL_visc > 0.5
32     warning('Viscous stability limit exceeded. Reduce dt or coarsen grid
33 .');
34 end
35
36 %% Initialise fields
37 [u, w, p, T] = initialise_fields(n_r, n_z);
38 [u, w, T] = apply_all_bc(u, w, T);
39
40 %% Factorise the Poisson operator (once)
41 % Hand-written DIRECT solver: block-Thomas elimination of the
42 % block-tridiagonal pressure operator, built on our own dense LU kernel
43 % (lu_factor.m / lu_solve.m). No MATLAB backslash, decomposition() or
44 % inv()
45 % appears anywhere in the solve path -- backslash is used only inside
46 % verify_poisson_solver.m, as an independent check. The operator is
47 % constant
48 % in time, so it is factorised once here; each time step then costs only
49 % the
50 % two sweeps in solve_poisson.m.
51 fprintf('\\nFactorising Poisson operator (block Thomas, direct) ...\\n');
52 t_fac = tic;
53 poisson_fac = factorize_poisson(r_c, r_f, dr, dz, n_r, n_z);
54 fprintf(' %d blocks of %d x %d -> %d unknowns (%.2f s)\\n', ...
55 n_z, n_r, n_r, n_r*n_z, toc(t_fac));
56

```

```

51 %% History arrays
52 hist_cont = zeros(n_steps, 1);
53 hist_vel  = zeros(n_steps, 1);
54 hist_temp = zeros(n_steps, 1);
55
56 %% Time loop
57 fprintf('\nStarting time integration ...\n');
58 converged_step = 0;
59 rhs_u_old = [];
60 rhs_w_old = [];
61 rhs_T_old = [];
62
63 for step = 1:n_steps
64
65     % AB2 predictor + one incremental pressure correction
66     [u_new, w_new, p_new, T_new, rhs_u, rhs_w, rhs_T] = ...
67         ab2_step(u, w, p, T, rhs_u_old, rhs_w_old, rhs_T_old, ...
68             r_c, r_f, dr, dz, dt, Re, Pr, n_r, n_z, ...
69             poisson_fac, alpha_p);
70
71     % Convergence check
72     res = compute_residuals(u_new, u, w_new, w, T_new, T, r_c, r_f, dr,
73         dz, dt);
74     hist_cont(step) = res.continuity;
75     hist_vel(step)  = res.momentum;
76     hist_temp(step) = res.temperature;
77
78     % Advance
79     u = u_new; w = w_new; p = p_new; T = T_new;
80     rhs_u_old = rhs_u;
81     rhs_w_old = rhs_w;
82     rhs_T_old = rhs_T;
83
84     % Progress print
85     if mod(step, c.output_interval) == 0
86         fprintf('  step %6d/%d  R_cont=%.2e  R_vel=%.2e  R_T=%.2e\n',
87             ...
88                 step, n_steps, res.continuity, res.momentum, res.
89                 temperature);
90         end
91
92         if check_convergence(res, c.tol_continuity, c.tol_velocity, c.
93             tol_temperature)
94             fprintf('\n  Converged at step %d!\n', step);
95             converged_step = step;
96             break;
97         end
98     end
99
100 if converged_step == 0
101     converged_step = n_steps;
102     warning('Maximum step count reached before all convergence
103         tolerances were met.');
```

```

104 hist_vel = hist_vel(1:converged_step);
105 hist_temp = hist_temp(1:converged_step);
106
107 %% Post-processing: map to cell centres for plotting
108 w_cc = 0.5 * (w(:, 1:end-1) + w(:, 2:end)); % (n_r, n_z)
109 u_cc = 0.5 * (u(1:end-1, :) + u(2:end, :)); % (n_r, n_z)
110
111 %% Output directory
112 plots_dir = fullfile(fileparts(mfilename('fullpath')), '..', '
    Plots_matlab');
113 if ~exist(plots_dir, 'dir')
114     mkdir(plots_dir);
115 end
116
117 %% Plot 1: axial velocity contour
118 light_figure();
119 contourf(z_c, r_c, w_cc, 20, 'LineStyle', 'none');
120 hold on;
121 % Decimated quiver layer so individual arrows stay readable; the
    underlying
122 % contour still contains every cell.
123 %
124 % Two corrections are needed because the axes are stretched ~100:1
125 % (z/D spans 50, r/D spans 0.5):
126 %
127 % 1. MATLAB scales quiver components in DATA units. A physically small
128 %     radial velocity ( $u/w \sim 0.06$  at most) then covers a large fraction
    of
129 %     the short r-axis and the arrows render almost vertically, which is
130 %     visually wrong for a flow that is almost entirely axial.
    Multiplying
131 %     the radial component by the axis aspect ratio makes the arrow's
132 %     *on-screen* angle equal to the true  $\text{atan}(u/w)$ .
133 %
134 % 2. Arrowhead size is likewise measured in data units, so the default
135 %     head puts wings tens of percent of the r-range wide -- they appear
    as
136 %     long diagonal streaks across the plot. The head is therefore
    scaled
137 %     down by the same aspect ratio.
138 z_span = 50.0;
139 r_span = 0.5;
140 aspect = r_span / z_span;
141
142 z_arrow = 1:8:length(z_c);
143 r_arrow = 1:3:length(r_c);
144 [Z_a, R_a] = meshgrid(z_c(z_arrow), r_c(r_arrow));
145
146 arrow_len = 1.6; % z/D length of the
    fastest arrow
147 a_scale = arrow_len / max(abs(w_cc(:)));
148
149 q = quiver(Z_a, R_a, ...
150     w_cc(r_arrow, z_arrow) * a_scale, ...
151     u_cc(r_arrow, z_arrow) * a_scale * aspect, ...
152     0, 'k', 'LineWidth', 0.5); % 0 -> autoscaling OFF
153 q.Clipping = 'on';
154 q.MaxHeadSize = 0.003;

```

```

155 xlim([0, 50]);
156 ylim([0, 0.5]);
157 cb = colorbar; cb.Label.String = 'w/W_{in}';
158 xlabel('z/D'); ylabel('r/D');
159 title('Axial velocity w/W_{in} and velocity vectors');
160 exportgraphics(gcf, fullfile(plots_dir, 'w_contour.png'), 'Resolution',
    200);
161 fprintf(' Saved w_contour.png\n');
162
163 %% Plot 2: radial velocity contour
164 % u is ~0 over most of the domain (flow is fully developed for z* > ~5,
165 % i.e. 90% of the pipe length) except for a strong, spatially localised
166 % entrance-corner value near (r=0.5, z=0). A linear color scale spanning
167 % the raw min/max is dominated by that single feature and renders the
168 % entire developed region as one flat color. Instead use a diverging
169 % colormap centred at zero with a robust (98th-percentile) symmetric
170 % limit, so the near-zero bulk keeps visible contrast and the entrance
171 % feature simply saturates the color scale rather than washing it out.
172 light_figure();
173 u_abs_sorted = sort(abs(u_cc(:)));
174 u_lim = u_abs_sorted(max(1, round(0.98 * numel(u_abs_sorted))));
175 if u_lim <= 0
176     u_lim = max(abs(u_cc(:)));
177 end
178 if u_lim <= 0
179     u_lim = 1;
180 end
181 contourf(z_c, r_c, u_cc, linspace(-u_lim, u_lim, 21), 'LineStyle', 'none
    ');
182 clim([-u_lim, u_lim]);
183 colorbar; colormap(gca, diverging_bwr(256)); xlabel('z/D'); ylabel('r/D'
    );
184 title('Radial velocity u/W_{in}');
185 exportgraphics(gcf, fullfile(plots_dir, 'u_contour.png'), 'Resolution',
    200);
186 fprintf(' Saved u_contour.png\n');
187
188 %% Plot 3: temperature contour
189 light_figure();
190 contourf(z_c, r_c, T, 20, 'LineStyle', 'none');
191 colorbar; colormap(gca, 'hot'); xlabel('z/D'); ylabel('r/D');
192 title('Non-dimensional temperature \theta');
193 exportgraphics(gcf, fullfile(plots_dir, 'theta_contour.png'), '
    Resolution', 200);
194 fprintf(' Saved theta_contour.png\n');
195
196 %% Plot 4: velocity profiles at selected z-positions
197 z_plot = [5, 10, 20, 30, 40, 50];
198 light_figure(); hold on;
199 w_an = analytical_velocity(r_c);
200 for k = 1:length(z_plot)
201     [~, jj] = min(abs(z_c - z_plot(k)));
202     plot(r_c, w_cc(:, jj), '-o', 'MarkerSize', 3, 'DisplayName', sprintf
        ('z*=%0f', z_plot(k)));
203 end
204 plot(r_c, w_an, 'k--', 'LineWidth', 1.5, 'DisplayName', 'Analytical');
205 xlabel('r/D'); ylabel('w/W_{in}');
206 title('Velocity profiles at selected z-positions');

```

```

207 legend('Location', 'best'); grid on;
208 exportgraphics(gcf, fullfile(plots_dir, 'w_profiles.png'), 'Resolution',
    200);
209 fprintf('  Saved w_profiles.png\n');
210
211 %% Plot 5: temperature profiles
212 light_figure(); hold on;
213 for k = 1:length(z_plot)
214     [~, jj] = min(abs(z_c - z_plot(k)));
215     plot(r_c, T(:, jj), '-o', 'MarkerSize', 3, 'DisplayName', sprintf('z
    *=%.0f', z_plot(k)));
216 end
217 % Analytical fully-developed profile at z*=50 (approved form; see
218 % report, Appendix B)
219 [~, j_Tfd] = min(abs(z_c - z_plot(end)));
220 z_Tfd = z_c(j_Tfd);
221 theta_an = analytical_temperature(r_c, z_Tfd, Re, Pr);
222 plot(r_c, theta_an, 'k--', 'LineWidth', 1.5, ...
    'DisplayName', sprintf('Analytical profile at z*=%.1f', z_Tfd));
223 xlabel('r/D'); ylabel('\theta');
224 title('Temperature profiles at selected z-positions');
225 legend('Location', 'best'); grid on;
226 exportgraphics(gcf, fullfile(plots_dir, 'T_profiles.png'), 'Resolution',
    200);
227 fprintf('  Saved T_profiles.png\n');
228
229
230 %% Plot 6: convergence history
231 light_figure();
232 semilogy(1:converged_step, hist_cont, 'b-', 'DisplayName', 'Continuity')
    ;
233 hold on;
234 semilogy(1:converged_step, hist_vel, 'r-', 'DisplayName', 'Momentum');
235 semilogy(1:converged_step, hist_temp, 'g-', 'DisplayName', 'Temperature'
    );
236 xlabel('Time step'); ylabel('Residual');
237 title('Convergence history');
238 legend('Location', 'best'); grid on;
239 exportgraphics(gcf, fullfile(plots_dir, 'convergence.png'), 'Resolution'
    , 200);
240 fprintf('  Saved convergence.png\n');
241
242 %% Plot 7: comparison with analytical at z*=50
243 [~, j_fd] = min(abs(z_c - 50.0));
244 w_num = w_cc(:, j_fd);
245 w_ana = analytical_velocity(r_c);
246
247 light_figure();
248 subplot(2,1,1);
249 plot(r_c, w_num, 'bo-', 'MarkerSize', 4, 'DisplayName', 'Numerical');
250 hold on;
251 plot(r_c, w_ana, 'r--', 'LineWidth', 1.5, 'DisplayName', 'Analytical');
252 xlabel('r/D'); ylabel('w/W_{in}');
253 title('Velocity at z*=50: Numerical vs Analytical');
254 legend('Location', 'best'); grid on;
255
256 subplot(2,1,2);
257 plot(r_c, abs(w_num - w_ana), 'k-', 'LineWidth', 1);
258 xlabel('r/D'); ylabel('|Error|');

```

```

259 title('Absolute error'); grid on;
260 exportgraphics(gcf, fullfile(plots_dir, 'w_comparison.png'), 'Resolution
    ', 200);
261 fprintf(' Saved w_comparison.png\n');
262
263 fprintf('\nAll plots saved to %s\n', plots_dir);
264
265 %% Save final fields for offline inspection / diagnostics
266 % Save next to THIS script. The path is derived from mfilename rather
    than
267 % from plots_dir, because the containing folder is called src_matlab in
    the
268 % development tree but matlab in the submission bundle -- hard-coding
269 % '../src_matlab' made this line error out there, after the plots were
270 % already written.
271 save(fullfile(fileparts(mfilename('fullpath'))), 'last_run.mat'), ...
272     'u', 'w', 'p', 'T', 'u_cc', 'w_cc', 'r_c', 'z_c', 'hist_cont', '
    hist_vel', 'hist_temp');
273
274 close all;

```

Listing 3: main.m — Main simulation loop and plotting

C.3 Grid Generation

```

1 function [r_c, r_f, z_c, z_f, dr, dz] = make_grid(n_r, n_z, L_over_D)
2 % MAKE_GRID Generate staggered grid for axisymmetric pipe flow.
3 %
4 % [r_c, r_f, z_c, z_f, dr, dz] = make_grid(n_r, n_z, L_over_D)
5 %
6 % Outputs (all non-dimensional, scaled by D):
7 % r_c : (n_r x 1) cell-centre radial coords [0 < r < 0.5]
8 % r_f : (n_r+1 x 1) r-face coords [0 <= r <= 0.5]
9 % z_c : (n_z x 1) cell-centre axial coords
10 % z_f : (n_z+1 x 1) z-face coords
11 % dr : radial spacing
12 % dz : axial spacing
13
14 dr = 0.5 / n_r;
15 dz = L_over_D / n_z;
16
17 r_f = linspace(0, 0.5, n_r + 1)'; % column vector
18 r_c = 0.5 * (r_f(1:end-1) + r_f(2:end));
19
20 z_f = linspace(0, L_over_D, n_z + 1)';
21 z_c = 0.5 * (z_f(1:end-1) + z_f(2:end));
22 end

```

Listing 4: make_grid.m — Staggered grid generation

C.4 Boundary Conditions

```

1 function [u, w, T] = apply_all_bc(u, w, T)
2 % APPLY_ALL_BC Apply all boundary conditions in one call.
3

```

```

4     u = apply_bc_u(u);
5     w = apply_bc_w(w);
6     T = apply_bc_T(T);
7 end

```

Listing 5: apply_all_bc.m — Driver applying all field boundary conditions

```

1 function w = apply_bc_w(w)
2 % APPLY_BC_W Enforce axial velocity boundary conditions.
3 %
4 %   w(:, 1)    = 1    inlet: uniform w* = 1
5 %   w(:, end) = w(:, end-1) outlet: zero-gradient
6
7     w(:, 1)    = 1.0;           % inlet
8     w(:, end) = w(:, end-1);   % outlet zero-gradient
9 end

```

Listing 6: apply_bc_w.m — Axial velocity boundary conditions

```

1 function u = apply_bc_u(u)
2 % APPLY_BC_U Enforce radial velocity boundary conditions.
3 %
4 %   u(1, :)    = 0    centreline (r = 0, symmetry)
5 %   u(end, :)  = 0    wall (r = R, no penetration)
6 %
7 %   NOTE: u(:,1) is at z_c(1), NOT the inlet face.
8 %   The inlet BC u=0 at z=0 is handled by the ghost cell in the
9 %   predictor.
10
11     u(1, :)    = 0;    % axis
12     u(end, :)  = 0;    % wall
13 end

```

Listing 7: apply_bc_u.m — Radial velocity boundary conditions

```

1 function T = apply_bc_T(T)
2 % APPLY_BC_T No-op for T -- kept only so apply_all_bc has a uniform
3 %   interface across u, w, T.
4 %
5 %   Inlet (Dirichlet theta=0) and outlet (zero-gradient) are enforced
6 %   purely through ghost cells inside compute_rhs_T (T_ghost_in=-T(:,1),
7 %   T_ghost_out=T(:,end)) -- NOT by overwriting the T(:,1)/T(:,end) cell
8 %   values here. Those are cell centres a half-cell away from the actual
9 %   inlet/outlet faces, so clamping them directly would over-constrain
10 %   the PDE and is inconsistent with the ghost-cell treatment used
11 %   everywhere else (see compute_rhs_T.m and
12 %   Appendix B of the report for the wall BC derivation).
13 %
14 %   Wall heat flux BC (dtheta/dr = +Re*Pr) is likewise handled entirely
15 %   via ghost cells inside compute_rhs_T.m.
16 end

```

Listing 8: apply_bc_T.m — Temperature boundary conditions

C.5 Momentum Solver

```

1 function w_star = predictor_w(w, u, r_c, r_f, dr, dz, dt, Re)
2 % PREDICTOR_W Advance axial velocity by one explicit Euler step (no
3 % pressure). Thin wrapper around compute_rhs_w; the main time loop
  uses
4 % compute_rhs_w directly (via ab2_step) to build the AB2 update.
5 %
6 % Returns w_star : (n_r, n_z+1) predicted axial velocity (BCs NOT yet
  reapplied)
7
8 rhs = compute_rhs_w(w, u, r_c, r_f, dr, dz, Re);
9 n_z1 = size(w, 2);
10 n_z = n_z1 - 1;
11 w_star = w;
12 w_star(:, 2:n_z) = w(:, 2:n_z) + dt * rhs;
13 end

```

Listing 9: predictor_w.m — Axial velocity predictor

```

1 function u_star = predictor_u(u, w, r_c, r_f, dr, dz, dt, Re)
2 % PREDICTOR_U Advance radial velocity by one explicit Euler step (no
3 % pressure). Thin wrapper around compute_rhs_u; the main time loop
  uses
4 % compute_rhs_u directly (via ab2_step) to build the AB2 update.
5 %
6 % Returns u_star : (n_r+1, n_z)
7
8 rhs = compute_rhs_u(u, w, r_c, r_f, dr, dz, Re);
9 u_star = u;
10 u_star(2:end-1, :) = u(2:end-1, :) + dt * rhs;
11 end

```

Listing 10: predictor_u.m — Radial velocity predictor

```

1 function face_val = upwind2_face(phi_im1, phi_i, phi_ip1, phi_ip2, vel)
2 % UPWIND2_FACE 2nd-order upwind (linear-extrapolation / LUD) face value
  .
3 %
4 % face value = 1.5*phi_i - 0.5*phi_im1 if vel > 0 (upwind: low-
  index side)
5 %               = 1.5*phi_ip1 - 0.5*phi_ip2 if vel < 0 (upwind: high-
  index side)
6 %
7 % phi_im1, phi_i, phi_ip1, phi_ip2 : values at relative positions i-1,
  i,
8 %           i+1, i+2 with respect to the face sitting between cells i and i
  +1.
9 % vel : transport velocity at the face.
10 %
11 % Retains the upwind stability bias needed at high cell Peclet number
12 % while being formally 2nd-order accurate (unlike pure central
13 % differencing or 1st-order upwind).
14
15 face_val = (1.5 * phi_i - 0.5 * phi_im1) .* (vel > 0) ...
16           + (1.5 * phi_ip1 - 0.5 * phi_ip2) .* (vel <= 0);
17 end

```

Listing 11: upwind2_face.m — 2nd-order upwind (linear-extrapolation) face-value helper

```

1 function rhs = compute_rhs_w(w, u, r_c, r_f, dr, dz, Re)
2 % COMPUTE_RHS_W Explicit RHS of the axial-momentum equation (no
   pressure,
3 %   no time discretization): rhs = -conv_r - conv_z + visc_r + visc_z
4 %
5 %   Kept separate from time-stepping for the AB2 history update.
6 %
7 %   Convective fluxes use a 2nd-order upwind (linear-extrapolation / LUD
   )
8 %   scheme -- see upwind2_face.m -- retaining the upwind stability bias
   at
9 %   high cell Peclet number while being formally 2nd-order accurate,
10 %   matching the diffusive terms (2nd-order central differencing).
11 %
12 %   w : (n_r, n_z+1)   current axial velocity (BCs already applied)
13 %   u : (n_r+1, n_z)   current radial velocity (BCs already applied)
14 %   r_c, r_f, dr, dz, Re : as usual
15 %
16 %   Returns rhs : (n_r, n_z-1) interior-face RHS (excludes j=1 and j=
   n_z+1)
17
18   [n_r, n_z1] = size(w);
19   n_z = n_z1 - 1;
20
21   % ---- ghost extension in r for w (depth 1, used by diffusion)
   -----
22   w_ghost_wall = -w(end, :);
23   w_ghost_axis = w(1, :);
24   w_ext = [w_ghost_axis; w; w_ghost_wall];    % (n_r+2, n_z+1)
25
26   % ---- second ghost layer in r, needed by the 2nd-order-upwind
   stencil --
27   w_ghost_axis2 = w(2, :);
28   w_ghost_wall2 = -w(end-1, :);
29   w_ext2 = [w_ghost_axis2; w_ghost_axis; w; w_ghost_wall;
   w_ghost_wall2];    % (n_r+4, n_z+1)
30   % w_ext2(i+2, :) = w(i, :)
31
32   % ---- interior slice j=2..n_z
   -----
33   w_int = w(:, 2:n_z);    % (n_r, n_z-1)
34
35   % ---- u interpolated to z_f(j) for j=2..n_z
   -----
36   u_at_zf = 0.5 * (u(:, 1:end-1) + u(:, 2:end));    % (n_r+1, n_z-1)
37
38   % --- Radial convective flux (1/r) d(r*u*w)/dr ---
39   w_r_im2 = w_ext2(1:n_r,    2:n_z);
40   w_r_im1 = w_ext2(2:n_r+1,  2:n_z);
41   w_r_i    = w_ext2(3:n_r+2,  2:n_z);    % = w_int
42   w_r_ip1  = w_ext2(4:n_r+3,  2:n_z);
43   w_r_ip2  = w_ext2(5:n_r+4,  2:n_z);
44
45   u_E = u_at_zf(2:end, :);
46   w_E_upwind = upwind2_face(w_r_im1, w_r_i, w_r_ip1, w_r_ip2, u_E);
47
48   u_I = u_at_zf(1:end-1, :);

```

```

49     w_I_upwind = upwind2_face(w_r_im2, w_r_im1, w_r_i, w_r_ip1, u_I);
50
51     % --- Axial convective flux  $d(w^2)/dz$  ---
52     % w already has real values exactly at the inlet/outlet faces, so
53     % only
54     % one extra ghost point beyond each end is needed (constant
55     % extension).
56     w_zghost_before = w(:, 1);
57     w_zghost_after  = w(:, end);
58     w_zext1 = [w_zghost_before, w, w_zghost_after];    % (n_r, n_z+3)
59
60     w_z_jm2 = w_zext1(:, 1:n_z-1);
61     w_z_jm1 = w_zext1(:, 2:n_z);
62     w_z_j    = w_zext1(:, 3:n_z+1);    % = w_int
63     w_z_jp1 = w_zext1(:, 4:n_z+2);
64     w_z_jp2 = w_zext1(:, 5:n_z+3);
65
66     w_N = 0.5 * (w_z_j + w_z_jp1);
67     w_N_upwind = upwind2_face(w_z_jm1, w_z_j, w_z_jp1, w_z_jp2, w_N);
68
69     w_S = 0.5 * (w_z_jm1 + w_z_j);
70     w_S_upwind = upwind2_face(w_z_jm2, w_z_jm1, w_z_j, w_z_jp1, w_S);
71
72     % ---- reshaping for broadcasting
73     -----
74     rc = r_c;
75     rf_o = r_f(2:end);
76     rf_i = r_f(1:end-1);
77
78     % ---- convective fluxes (2nd-order upwind)
79     -----
80     conv_r = (rf_o .* u_E .* w_E_upwind - rf_i .* u_I .* w_I_upwind) ./
81     (rc .* dr);
82     conv_z = (w_N .* w_N_upwind - w_S .* w_S_upwind) / dz;
83
84     % ---- diffusive fluxes (central, 2nd order)
85     -----
86     visc_r = (rf_o .* (w_ext(3:end, 2:n_z) - w_int) ...
87             - rf_i .* (w_int - w_ext(1:end-2, 2:n_z))) ./ (rc * dr^2) /
88     Re;
89
90     visc_z = (w(:, 3:n_z+1) - 2*w_int + w(:, 1:n_z-1)) / dz^2 / Re;
91
92     rhs = -conv_r - conv_z + visc_r + visc_z;
93 end

```

Listing 12: compute_rhs_w.m — Axial momentum RHS: 2nd-order upwind convection, central diffusion

```

1 function rhs = compute_rhs_u(u, w, r_c, r_f, dr, dz, Re)
2 % COMPUTE_RHS_U Explicit RHS of the radial-momentum equation (no
3 % pressure,
4 % no time discretization). Reused by the AB2 time integrator.
5 % Convective fluxes use a 2nd-order upwind (linear-extrapolation / LUD
6 % scheme -- see upwind2_face.m.
7 %

```

```

8 % u : (n_r+1, n_z)
9 % w : (n_r, n_z+1)
10 % r_c, r_f, dr, dz, Re : as usual
11 %
12 % Returns rhs : (n_r-1, n_z) interior-face RHS (excludes i=1 and i=
n_r+1)
13
14 [n_rpl, n_z] = size(u);
15 n_r = n_rpl - 1;
16
17 % ---- ghost extension in z for u (depth 2), needed by diffusion and
by --
18 % the 2nd-order-upwind stencil.
19 u_ghost_in1 = -u(:, 1);
20 u_ghost_in2 = -u(:, 2);
21 u_ghost_out1 = u(:, end);
22 u_ghost_out2 = u(:, end);
23 u_ext = [u_ghost_in1, u, u_ghost_out1]; % (
n_r+1, n_z+2), depth 1 (diffusion)
24 u_zext2 = [u_ghost_in2, u_ghost_in1, u, u_ghost_out1, u_ghost_out2];
% (n_r+1, n_z+4)
25
26 % ---- second ghost layer in r, needed by the 2nd-order-upwind
stencil ---
27 u_ghost_axis_ext = -u(2, :); % represents "u(0)" (one before
axis)
28 u_ghost_wall_ext = -u(end-1, :); % represents "u(n_r+2)" (one
beyond wall)
29 u_ext_r = [u_ghost_axis_ext; u; u_ghost_wall_ext]; % (n_r+3, n_z)
30
31 % ---- interior slice i=2..n_r
-----
32 u_int = u(2:end-1, :); % (n_r-1, n_z)
33
34 % --- Radial convective flux (1/r) d(r*u^2)/dr ---
35 u_r_im2 = u_ext_r(1:n_r-1, :);
36 u_r_im1 = u_ext_r(2:n_r, :);
37 u_r_i = u_ext_r(3:n_r+1, :); % = u_int
38 u_r_ip1 = u_ext_r(4:n_r+2, :);
39 u_r_ip2 = u_ext_r(5:n_r+3, :);
40
41 u_out = 0.5 * (u_int + u(3:end, :));
42 u_in = 0.5 * (u(1:end-2, :) + u_int);
43
44 u_out_upwind = upwind2_face(u_r_im1, u_r_i, u_r_ip1, u_r_ip2, u_out)
;
45 u_in_upwind = upwind2_face(u_r_im2, u_r_im1, u_r_i, u_r_ip1, u_in);
46
47 % ---- face velocities for axial convection
-----
48 w_at_rf = 0.5 * (w(1:end-1, :) + w(2:end, :)); % (n_r-1, n_z+1)
49 w_top = w_at_rf(:, 2:end);
50 w_bot = w_at_rf(:, 1:end-1);
51
52 % --- Axial convective flux d(u*w)/dz --- (2nd-order upwind,
interior rows)
53 u_z_jm2 = u_zext2(2:end-1, 1:n_z);
54 u_z_jm1 = u_zext2(2:end-1, 2:n_z+1);

```

```

55     u_z_j    = u_zext2(2:end-1, 3:n_z+2);    % = u_int
56     u_z_jp1 = u_zext2(2:end-1, 4:n_z+3);
57     u_z_jp2 = u_zext2(2:end-1, 5:n_z+4);
58
59     u_top_upwind = upwind2_face(u_z_jm1, u_z_j, u_z_jp1, u_z_jp2, w_top)
60 ;
61     u_bot_upwind = upwind2_face(u_z_jm2, u_z_jm1, u_z_j, u_z_jp1, w_bot)
62 ;
63
64     % ---- reshaping for broadcasting
65     -----
66     rc_o    = r_c(2:end);
67     rc_i    = r_c(1:end-1);
68     rf_int  = r_f(2:end-1);
69
70     % ---- convective fluxes (2nd-order upwind)
71     -----
72     conv_r = (rc_o .* u_out .* u_out_upwind - rc_i .* u_in .*
73 u_in_upwind) ./ (rf_int .* dr);
74     conv_z = (w_top .* u_top_upwind - w_bot .* u_bot_upwind) / dz;
75
76     % ---- diffusive fluxes (central, 2nd order)
77     -----
78     visc_r = ((rc_o .* (u(3:end, :) - u_int) - rc_i .* (u_int - u(1:end
79 -2, :))) ./ (rf_int * dr^2) ...
80             - u_int ./ rf_int.^2) / Re;
81
82     visc_z = (u_ext(2:end-1, 3:end) - 2*u_int + u_ext(2:end-1, 1:end-2))
83 / dz^2 / Re;
84
85     rhs = -conv_r - conv_z + visc_r + visc_z;
86 end

```

Listing 13: compute_rhs_u.m — Radial momentum RHS: 2nd-order upwind convection, central diffusion

```

1 function [u_new, w_new, p_new, T_new, rhs_u, rhs_w, rhs_T] = ...
2     ab2_step(u, w, p, T, rhs_u_old, rhs_w_old, rhs_T_old, ...
3         r_c, r_f, dr, dz, dt, Re, Pr, n_r, n_z, poisson_fac,
4         alpha_p)
5 % AB2_STEP Adams-Bashforth 2 with one incremental pressure correction.
6 % The first call uses forward Euler when the previous RHS arrays are
7 % empty.
8
9     rhs_u = compute_rhs_u(u, w, r_c, r_f, dr, dz, Re);
10    rhs_w = compute_rhs_w(w, u, r_c, r_f, dr, dz, Re);
11    rhs_T = compute_rhs_T(T, u, w, r_c, r_f, dr, dz, Re, Pr);
12
13    if isempty(rhs_u_old)
14        explicit_u = rhs_u;
15        explicit_w = rhs_w;
16        explicit_T = rhs_T;
17    else
18        explicit_u = 1.5 * rhs_u - 0.5 * rhs_u_old;
19        explicit_w = 1.5 * rhs_w - 0.5 * rhs_w_old;
20        explicit_T = 1.5 * rhs_T - 0.5 * rhs_T_old;
21    end

```

```

21 % The old pressure gradient makes the Poisson unknown a correction p
22 ,
23 u_star = u;
24 grad_p_r = (p(2:end, :) - p(1:end-1, :)) / dr;
25 u_star(2:end-1, :) = u(2:end-1, :) ...
26     + dt * (explicit_u - grad_p_r);
27
28 w_star = w;
29 grad_p_z = (p(:, 2:end) - p(:, 1:end-1)) / dz;
30 w_star(:, 2:end-1) = w(:, 2:end-1) ...
31     + dt * (explicit_w - grad_p_z);
32 u_star = apply_bc_u(u_star);
33 w_star = apply_bc_w(w_star);
34
35 p_prime = solve_poisson(poisson_fac, u_star, w_star, ...
36     r_c, r_f, dr, dz, dt, n_r, n_z);
37 [u_new, w_new] = correct_velocity(u_star, w_star, ...
38     p_prime, dr, dz, dt);
39 u_new = apply_bc_u(u_new);
40 w_new = apply_bc_w(w_new);
41
42 p_new = update_pressure(p, p_prime, alpha_p);
43 T_new = T + dt * explicit_T;
end

```

Listing 14: ab2_step.m — AB2 step with one incremental pressure correction

C.6 Poisson Solver and Pressure Correction

```

1 function [LU, piv] = lu_factor(A)
2 % LU_FACTOR Doolittle LU factorisation with partial pivoting: P*A = L*
3 % U.
4 % [LU, piv] = lu_factor(A)
5 %
6 % The assignment requires a DIRECT solution method for the pressure
7 % Poisson
8 % equation that is programmed, not a library call. This routine and
9 % LU_SOLVE are that elimination kernel, written out explicitly -- no
10 % backslash, no lu(), no inv(), no decomposition(). They act on the
11 % small
12 % (n_r x n_r) blocks of the block-tridiagonal Poisson operator and are
13 % called only during set-up, from FACTORIZE_POISSON.
14 %
15 % Inputs:
16 % A : (n x n) dense matrix (not modified)
17 %
18 % Outputs:
19 % LU : (n x n) packed factors. The strict lower triangle holds L (
20 % its
21 % unit diagonal is implied); the upper triangle including the
22 % diagonal holds U.
23 % piv : (n x 1) piv(k) is the row that was interchanged with row k
24 % at
25 % elimination step k.
26 %
27 % See also LU_SOLVE, FACTORIZE_POISSON, SOLVE_POISSON.

```

```

24
25     n = size(A, 1);
26     LU = A;
27     piv = zeros(n, 1);
28
29     for k = 1:n
30         % --- partial pivoting: largest magnitude in column k, rows k..n
31         [pivot_val, m] = max(abs(LU(k:n, k)));
32         m = m + k - 1;
33         piv(k) = m;
34         if pivot_val == 0
35             error('lu_factor:singular', 'Zero pivot in column %d.', k);
36         end
37         if m ~= k
38             LU([k m], :) = LU([m k], :);
39         end
40
41         % --- elimination; the multipliers are stored in the created
42         zeros
43         rows = (k+1):n;
44         LU(rows, k) = LU(rows, k) / LU(k, k);
45         LU(rows, (k+1):n) = LU(rows, (k+1):n) - LU(rows, k) * LU(k, (k
46         +1):n);
47     end
48 end

```

Listing 15: lu_factor.m — Doolittle LU with partial pivoting (elimination kernel)

```

1 function X = lu_solve(LU, piv, B)
2 % LU_SOLVE Solve A*X = B from the factors produced by LU_FACTOR.
3 %
4 % X = lu_solve(LU, piv, B)
5 %
6 % Forward substitution with the unit lower triangle L, then back
7 % substitution with U. Written out explicitly -- no backslash.
8 %
9 % Inputs:
10 % LU, piv : output of LU_FACTOR
11 % B       : (n x 1) or (n x m) one or several right-hand sides
12 %
13 % Output:
14 % X       : same size as B
15 %
16 % See also LU_FACTOR, FACTORIZE_POISSON.
17
18 n = size(LU, 1);
19 X = B;
20
21 % --- apply the recorded row interchanges: b <- P*b
22 for k = 1:n
23     m = piv(k);
24     if m ~= k
25         X([k m], :) = X([m k], :);
26     end
27 end
28
29 % --- forward substitution L*Y = P*b (L has unit diagonal)
30 for k = 1:(n-1)

```

```

31     X((k+1):n, :) = X((k+1):n, :) - LU((k+1):n, k) * X(k, :);
32 end
33
34 % --- back substitution  U*X = Y
35 for k = n:-1:1
36     X(k, :) = X(k, :) / LU(k, k);
37     X(1:(k-1), :) = X(1:(k-1), :) - LU(1:(k-1), k) * X(k, :);
38 end
39 end

```

Listing 16: lu_solve.m — Forward/back substitution

```

1 function P = factorize_poisson(r_c, r_f, dr, dz, n_r, n_z)
2 % FACTORIZE_POISSON Block-Thomas factorisation of the pressure Poisson
3 % operator, using the hand-written LU kernel in LU_FACTOR / LU_SOLVE.
4 % Called ONCE, before the time loop.
5 %
6 % P = factorize_poisson(r_c, r_f, dr, dz, n_r, n_z)
7 %
8 % STRUCTURE
9 % -----
10 % Group the unknowns p'(:, j) by axial station j. The 5-point stencil
11 % couples a cell only to its radial neighbours (same j) and to j +/-
12 % 1, so
13 % the operator is BLOCK TRIDIAGONAL with n_z blocks of size n_r x n_r:
14 %
15 %  $S_j * p_{j-1} + D_j * p_j + N_j * p_{j+1} = b_j$  ,  $j = 1 \dots n_z$ 
16 %
17 %  $D_j = \text{tridiag}(w_i, -(e_i + w_i + 2*a_z), e_i)$  (interior
18 % j)
19 %
20 %  $S_j = N_j = a_z * I$  ,  $a_z = 1/dz^2$ 
21 %  $e_i = r_{i+1/2} / (r_i * dr^2)$  ,  $w_i = r_{i-1/2} / (r_i * dr^2)$ 
22 %
23 % The boundary conditions enter as:
24 %
25 % Wall (i = n_r):  $e_{n_r} = 0$  (
26 % Neumann)
27 %
28 % Centreline (i = 1):  $w_1 = 0$  automatically,  $r_{1/2} = 0$  (
29 % Neumann)
30 %
31 % Inlet (j = 1): no south coupling, so that diagonal carries
32 %  $a_z$  once instead of twice (
33 % Neumann)
34 %
35 % Outlet (j = n_z):  $D = I, S = N = 0$  (
36 % Dirichlet)
37 %
38 % BLOCK THOMAS FORWARD ELIMINATION
39 % -----
40 %
41 %  $M_1 = D_1$ 
42 %  $M_j = D_j - S_j * \text{inv}(M_{j-1}) * N_{j-1} = D_j - a_z^2 * \text{inv}(M_{j-1})$ 
43 %
44 % which reduces the solve to the two sweeps in SOLVE_POISSON,
45 %  $y_j = \text{inv}(M_j) * (b_j - a_z * y_{j-1})$  and  $p_j = y_j - a_z * \text{inv}(M_j) * p_{j+1}$ .
46 %
47 %
48 % The operator is constant in time, so the n_z Schur complements are
49 % inverted here, once, via our own LU. Every subsequent Poisson solve
50 % then
51 % costs only two (n_r x n_r) matrix-vector products per axial station,
52 % i.e.

```

```

38 %  $O(n_r^2 n_z)$  -- cheaper than re-factorising anything.
39 %
40 % STABILITY
41 % -----
42 % -A is an irreducibly diagonally dominant M-matrix: |diagonal| equals
43 % the sum of the off-diagonal magnitudes in every interior row and is
44 % strictly greater in the Dirichlet outlet rows. Every Schur complement  $M_j$ 
45 % inherits that property, so no pivoting is needed between blocks. Partial
46 % pivoting is used inside LU_FACTOR regardless.
47 %
48 % Output:
49 %   P.Minv : ( $n_r \times n_r \times n_z$ ) the inverted Schur complements  $\text{inv}(M_j)$ 
50 %   P.az   :  $1/dz^2$ 
51 %   P.n_r, P.n_z
52 %
53 % See also SOLVE_POISSON, LU_FACTOR, LU_SOLVE, VERIFY_POISSON_SOLVER.
54
55 az = 1.0 / dz^2;
56
57 % --- radial stencil coefficients (independent of j)
58 -----
59 e = r_f(2:end) ./ (r_c * dr^2); % coupling to i+1
60 w = r_f(1:end-1) ./ (r_c * dr^2); % coupling to i-1; w(1)=0 as r_f
61 (1)=0
62 e(n_r) = 0.0; % Neumann at the wall
63
64 off_super = diag(e(1:n_r-1), 1);
65 off_sub = diag(w(2:n_r), -1);
66
67 D_inlet = diag(-(e + w + az)) + off_super + off_sub; % j = 1
68 D_int = diag(-(e + w + 2.0 * az)) + off_super + off_sub; % 2 <= j
69 <= n_z-1
70
71 I_n = eye(n_r);
72 Minv = zeros(n_r, n_r, n_z);
73
74 % --- j = 1: no south coupling, so  $M_1 = D_1$ 
75 -----
76 [LU, piv] = lu_factor(D_inlet);
77 Minv(:, :, 1) = lu_solve(LU, piv, I_n);
78
79 % --- j = 2 .. n_z-1: Schur complement update, then invert
80 -----
81 for j = 2:(n_z - 1)
82     [LU, piv] = lu_factor(D_int - az^2 * Minv(:, :, j-1));
83     Minv(:, :, j) = lu_solve(LU, piv, I_n);
84 end
85
86 % --- j = n_z: Dirichlet row,  $D = I$  and no south coupling
87 -----
88 Minv(:, :, n_z) = I_n;
89
90 P = struct('Minv', Minv, 'az', az, 'n_r', n_r, 'n_z', n_z);
91 end

```

Listing 17: factorize_poisson.m — Block-Thomas factorisation of the Poisson operator (called once)

```

1 function p_prime = solve_poisson(P, u_star, w_star, r_c, r_f, dr, dz, dt
  , n_r, n_z)
2 % SOLVE_POISSON Solve lap(p') = (1/dt)*div(u*) for the pressure
  correction,
3 % by the block-Thomas sweeps prepared in FACTORIZE_POISSON.
4 %
5 % p_prime = solve_poisson(P, u_star, w_star, r_c, r_f, dr, dz, dt, n_r
  , n_z)
6 %
7 % This is a DIRECT method, programmed here rather than delegated: no
8 % backslash, no decomposition(), no inv(). With the unknowns grouped
  by
9 % axial station, column j of the (n_r x n_z) field IS block j of the
10 % block-tridiagonal system, so no flattening or flat-index bookkeeping
  is
11 % needed anywhere.
12 %
13 % forward : y_j = inv(M_j)*(b_j - a_z*y_{j-1})
14 % back : p_j = y_j - a_z*inv(M_j)*p_{j+1}
15 %
16 % Inputs:
17 % P : struct from FACTORIZE_POISSON (built once, before the
  time loop)
18 % u_star : (n_r+1 x n_z) predicted radial velocity
19 % w_star : (n_r x n_z+1) predicted axial velocity
20 %
21 % Output:
22 % p_prime : (n_r x n_z)
23 %
24 % See also FACTORIZE_POISSON, LU_FACTOR, LU_SOLVE.
25
26 % ---- right-hand side: b(i,j) = (1/dt)*div(u*)
  -----
27 rf_o = r_f(2:end); % (n_r, 1)
28 rf_i = r_f(1:end-1); % (n_r, 1)
29 rc = r_c; % (n_r, 1)
30
31 % Radial divergence
32 div_u = (rf_o .* u_star(2:end, :) - rf_i .* u_star(1:end-1, :)) ./ (
  rc * dr);
33
34 % Axial divergence
35 div_w = (w_star(:, 2:end) - w_star(:, 1:end-1)) / dz;
36
37 b = (div_u + div_w) / dt; % (n_r, n_z)
38
39 % Outlet column (j = n_z) has Dirichlet p' = 0 -> RHS = 0
40 b(:, end) = 0.0;
41
42 % ---- forward sweep
  -----
43 Minv = P.Minv;
44 az = P.az;
45

```

```

46 y = zeros(n_r, n_z);
47 y(:, 1) = Minv(:, :, 1) * b(:, 1);
48 for j = 2:(n_z - 1)
49     y(:, j) = Minv(:, :, j) * (b(:, j) - az * y(:, j-1));
50 end
51 y(:, n_z) = b(:, n_z);          % Dirichlet row: M = I, no south
coupling
52
53 % ---- back substitution
-----
54 p_prime = zeros(n_r, n_z);
55 p_prime(:, n_z) = y(:, n_z);
56 for j = (n_z - 1):-1:1
57     p_prime(:, j) = y(:, j) - az * (Minv(:, :, j) * p_prime(:, j+1))
;
58 end
59 end

```

Listing 18: solve_poisson.m — Poisson solve by the two block-Thomas sweeps

```

1 function A = build_poisson_matrix(r_c, r_f, dr, dz, n_r, n_z)
2 % BUILD_POISSON_MATRIX Assemble sparse Poisson matrix (called once).
3 %
4 % A = build_poisson_matrix(r_c, r_f, dr, dz, n_r, n_z)
5 %
6 % Returns A : sparse matrix (n_r*n_z x n_r*n_z)
7 %
8 % Flat index: k = (i-1)*n_z + j (1-based)
9 %
10 % BCs for p':
11 %   Centreline (i=1):      Neumann (r_f(1)=0, flux vanishes naturally)
12 %   Wall (i=n_r):         Neumann (outer flux = 0)
13 %   Inlet (j=1):          Neumann (south flux = 0)
14 %   Outlet (j=n_z):       Dirichlet p' = 0
15
16 N = n_r * n_z;
17
18 % Pre-allocate triplets (at most 5 entries per row)
19 rows = zeros(5*N, 1);
20 cols = zeros(5*N, 1);
21 vals = zeros(5*N, 1);
22 idx = 0;
23
24 for i = 1:n_r
25     for j = 1:n_z
26         k = (i-1)*n_z + j;
27
28         % Dirichlet at outlet (j = n_z): p' = 0
29         if j == n_z
30             idx = idx + 1;
31             rows(idx) = k; cols(idx) = k; vals(idx) = 1.0;
32             continue;
33         end
34
35         % Coefficients from cylindrical Poisson
36         rf_o = r_f(i + 1); % r_{i+1/2}
37         rf_i_val = r_f(i); % r_{i-1/2}
38         rc = r_c(i);

```

```

39         coeff_E = rf_o / (rc * dr^2);           % p'(i+1, j)
40         coeff_W = rf_i_val / (rc * dr^2);       % p'(i-1, j)
41         coeff_N = 1.0 / dz^2;                   % p'(i, j+1)
42         coeff_S = 1.0 / dz^2;                   % p'(i, j-1)
43
44
45         % Neumann wall (i = n_r): outer flux = 0
46         if i == n_r
47             coeff_E = 0.0;
48         end
49
50         % Neumann inlet (j = 1): south flux = 0
51         if j == 1
52             coeff_S = 0.0;
53         end
54
55         % Centre coefficient
56         coeff_C = -(coeff_E + coeff_W + coeff_N + coeff_S);
57
58         % Diagonal
59         idx = idx + 1;
60         rows(idx) = k; cols(idx) = k; vals(idx) = coeff_C;
61
62         % East (i+1, j)
63         if i < n_r
64             idx = idx + 1;
65             rows(idx) = k; cols(idx) = i*n_z + j; vals(idx) =
coeff_E;
66         end
67
68         % West (i-1, j)
69         if i > 1
70             idx = idx + 1;
71             rows(idx) = k; cols(idx) = (i-2)*n_z + j; vals(idx) =
coeff_W;
72         end
73
74         % North (i, j+1)
75         if j < n_z
76             idx = idx + 1;
77             rows(idx) = k; cols(idx) = (i-1)*n_z + (j+1); vals(idx)
= coeff_N;
78         end
79
80         % South (i, j-1)
81         if j > 1
82             idx = idx + 1;
83             rows(idx) = k; cols(idx) = (i-1)*n_z + (j-1); vals(idx)
= coeff_S;
84         end
85     end
86 end
87
88 % Trim and build sparse matrix
89 rows = rows(1:idx);
90 cols = cols(1:idx);
91 vals = vals(1:idx);
92 A = sparse(rows, cols, vals, N, N);

```

93 end

Listing 19: build_poisson_matrix.m — Sparse assembly of the same operator, used only for verification

```
1 %% VERIFY_POISSON_SOLVER Verification of the hand-written direct
  Poisson solver.
2 %
3 % The assignment asks for a direct solution method that is PROGRAMMED,
  not a
4 % library call. FACTORIZE_POISSON / SOLVE_POISSON implement the block
  Thomas
5 % algorithm on top of our own Doolittle LU kernel (LU_FACTOR / LU_SOLVE
  ).
6 % This script checks that implementation:
7 %
8 % [0] the dense LU kernel on a small random system;
9 % [1] the backward error of the block-Thomas solution, i.e. the
  relative
10 % residual ||A*p' - b||_inf / ||b||_inf against the independently
11 % assembled sparse operator A from BUILD_POISSON_MATRIX. This is
  the
12 % meaningful accuracy measure: the operator is stiff (cond_1(A) ~
  1e8),
13 % so the *solution* difference against any other solver is
  bounded by
14 % cond(A)*eps and says little about either solver;
15 % [2] agreement with MATLAB's sparse direct solver (backslash) on the
  same
16 % system, as an independent reference. This is the ONLY place in
  the
17 % whole code where backslash appears -- it is a check, not the
  solver;
18 % [3] that the Dirichlet outlet condition p' = 0 comes out as imposed
  .
19 %
20 % Run: verify_poisson_solver
21
22 clear; clc;
23
24 c = constants();
25 n_r = c.n_r; n_z = c.n_z; dt = c.dt;
26 [r_c, r_f, ~, ~, dr, dz] = make_grid(n_r, n_z, c.L_over_D);
27
28 fprintf('=====\\
  n');
29 fprintf('Verification of the hand-written direct Poisson solver\\n');
30 fprintf(' grid: n_r = %d, n_z = %d (%d unknowns)\\n', n_r, n_z, n_r*
  n_z);
31 fprintf(' block structure: %d blocks of %d x %d\\n', n_z, n_r, n_r);
32 fprintf('=====\\
  n');
33
34 %% [0] the dense LU kernel itself
35 rng(42);
36 M = randn(12);
37 rhs = randn(12, 1);
38 [LU, piv] = lu_factor(M);
```

```

39 x = lu_solve(LU, piv, rhs);
40 fprintf('\n[0] dense LU kernel, 12x12 random system\n');
41 fprintf('      ||M*x - rhs||_inf = %.3e\n', max(abs(M*x - rhs)));
42
43 %% [1] factorise and solve
44 t0 = tic;
45 P = factorize_poisson(r_c, r_f, dr, dz, n_r, n_z);
46 t_fac = toc(t0);
47 fprintf('\n[1] block-Thomas factorisation: %.1f ms (once)\n', t_fac*1e3)
48 ;
49 % A divergence-carrying velocity predictor with the wall/axis BCs of u
50 rng(0);
51 u_star = randn(n_r+1, n_z);
52 u_star(1, :) = 0.0; % symmetry axis
53 u_star(end, :) = 0.0; % no-penetration at the wall
54 w_star = randn(n_r, n_z+1);
55
56 n_rep = 20;
57 t0 = tic;
58 for k = 1:n_rep
59     p_prime = solve_poisson(P, u_star, w_star, r_c, r_f, dr, dz, dt, n_r
60 , n_z);
61 end
62 t_solve = toc(t0) / n_rep;
63 % Rebuild the same RHS that solve_poisson forms internally
64 div_u = (r_f(2:end) .* u_star(2:end, :) - r_f(1:end-1) .* u_star(1:end
65 -1, :)) ./ (r_c * dr);
66 div_w = (w_star(:, 2:end) - w_star(:, 1:end-1)) / dz;
67 b = (div_u + div_w) / dt;
68 b(:, end) = 0.0;
69 % Flat ordering used by build_poisson_matrix: k = (i-1)*n_z + j
70 b_flat = reshape(b', [], 1);
71 p_flat = reshape(p_prime', [], 1);
72
73 A = build_poisson_matrix(r_c, r_f, dr, dz, n_r, n_z);
74 res_own = max(abs(A*p_flat - b_flat)) / max(abs(b_flat));
75 fprintf('      solve: %.3f ms per Poisson solve\n', t_solve*1e3);
76 fprintf('      relative residual ||A p'' - b||_inf / ||b||_inf = %.3e\n',
77     res_own);
78 %% [2] independent reference: MATLAB sparse backslash (check only)
79 p_ref_flat = A \ b_flat;
80 res_ref = max(abs(A*p_ref_flat - b_flat)) / max(abs(b_flat));
81 d_rel = max(abs(p_flat - p_ref_flat)) / max(abs(p_ref_flat));
82 cond_est = condest(A);
83 fprintf('\n[2] reference MATLAB backslash (used here only, as a check)\n
84 ');
85 fprintf('      relative residual, backslash                = %.3e\n',
86     res_ref);
87 fprintf('      ||p''_own - p''_backslash||_inf / ||p''||_inf      = %.3e\n
88 ', d_rel);
89 fprintf('      cond_1(A) estimate                                = %.3e\n',
90     cond_est);
91 fprintf('      cond_1(A) * eps                                    = %.3e (
92     bound on the line above)\n', ...

```

```

88         cond_est * eps);
89
90 %% [3] boundary conditions
91 fprintf('\n[3] boundary conditions\n');
92 fprintf('      max |p''| on the outlet column (Dirichlet p''=0) = %.3e\n',
93         , ...
94         max(abs(p_prime(:, end)))));
95
96 %% Verdict
97 fprintf('\n
98 =====\n');
99 if res_own < 1e-10
100     fprintf('PASS - hand-written direct solver reproduces the operator\n
101 ');
102 else
103     fprintf('FAIL - residual too large\n');
104 end
105 fprintf('=====
106 \n');

```

Listing 20: verify_poisson_solver.m — Verification of the hand-written direct solver

```

1 function div = compute_divergence(u, w, r_c, r_f, dr, dz)
2 % COMPUTE_DIVERGENCE Pointwise discrete divergence of velocity field.
3 %
4 %   div(i,j) = [r_{i+1/2}*u(i+1,j) - r_{i-1/2}*u(i,j)] / (r_c(i)*dr)
5 %               + [w(i,j+1) - w(i,j)] / dz
6 %
7 %   Returns div : (n_r, n_z)
8
9   rf_o = r_f(2:end);      % (n_r, 1)
10  rf_i = r_f(1:end-1);    % (n_r, 1)
11  rc   = r_c;             % (n_r, 1)
12
13  div_u = (rf_o .* u(2:end, :) - rf_i .* u(1:end-1, :)) ./ (rc * dr);
14  div_w = (w(:, 2:end) - w(:, 1:end-1)) / dz;
15
16  div = div_u + div_w;
17 end

```

Listing 21: compute_divergence.m — Discrete velocity divergence (Poisson RHS and continuity residual)

```

1 function [u_new, w_new] = correct_velocity(u_star, w_star, p_prime, dr,
2      dz, dt)
3 % CORRECT_VELOCITY Apply pressure correction to obtain div-free
4 %   velocities.
5 %
6 %   u^{n+1}(i,j) = u*(i,j) - dt * (p'(i,j) - p'(i-1,j)) / dr
7 %   w^{n+1}(i,j) = w*(i,j) - dt * (p'(i,j) - p'(i,j-1)) / dz
8
9   u_new = u_star;
10  w_new = w_star;
11
12  % Radial velocity at interior r-faces (rows 2:end-1)
13  u_new(2:end-1, :) = u_star(2:end-1, :) ...
14      - dt * (p_prime(2:end, :) - p_prime(1:end-1, :)) / dr;

```

```

14 % Axial velocity at interior z-faces (columns 2:end-1)
15 w_new(:, 2:end-1) = w_star(:, 2:end-1) ...
16     - dt * (p_prime(:, 2:end) - p_prime(:, 1:end-1)) / dz;
17 end

```

Listing 22: correct_velocity.m — Velocity correction step

```

1 function p_new = update_pressure(p, p_prime, zeta)
2 % UPDATE_PRESSURE Accumulate relaxed correction: p_new = p + zeta*
   p_prime.
3
4     p_new = p + zeta * p_prime;
5 end

```

Listing 23: update_pressure.m — Additive pressure-correction update

C.7 Temperature Solver

```

1 function rhs = compute_rhs_T(T, u, w, r_c, r_f, dr, dz, Re, Pr)
2 % COMPUTE_RHS_T Explicit RHS of the energy equation (no time
3 % discretization): rhs = -conv_r - conv_z + visc_r + visc_z
4 %
5 % Reused as the current and stored previous RHS in ab2_step.m.
6 %
7 % Convective fluxes use a 2nd-order upwind (linear-extrapolation / LUD
   )
8 % scheme -- see upwind2_face.m. Diffusive terms use standard 2nd-order
9 % central differencing.
10 %
11 % Wall BC: dtheta/dr* = +Re*Pr (see report, Appendix B,
12 % for the derivation -- NOT -1, which drops the Re*Pr factor from the
13 % non-dimensionalization).
14 % Implemented as a ghost cell: theta_ghost(n_r+1,:) = theta(n_r,:) +
   dr*Re*Pr
15 %
16 % T : (n_r, n_z) current theta (BCs already applied)
17 % u : (n_r+1, n_z) radial velocity at r-faces
18 % w : (n_r, n_z+1) axial velocity at z-faces
19 % r_c, r_f, dr, dz, Re, Pr : as usual
20 %
21 % Returns rhs : (n_r, n_z)
22
23 [n_r, n_z] = size(T);
24 alpha = 1.0 / (Re * Pr);
25 RePr = Re * Pr;
26
27 % ---- ghost extension in r (depth 1, used by diffusion)
   -----
28 T_ghost_axis = T(1, :);
29 T_ghost_wall = T(end, :) + dr * RePr;
30 T_ext = [T_ghost_axis; T; T_ghost_wall]; % (n_r+2, n_z)
31
32 % ---- ghost extension in z (depth 1, used by diffusion)
   -----
33 T_ghost_in = -T(:, 1);
34 T_ghost_out = T(:, end);
35 T_zext = [T_ghost_in, T, T_ghost_out]; % (n_r, n_z+2)

```

```

36
37 % ---- second ghost layer, needed by the 2nd-order-upwind stencil
38 -----
39 T_ghost_axis2 = T(2, :);
40 T_ghost_wall2 = T(end, :) + 2.0 * dr * RePr;
41 T_ext2 = [T_ghost_axis2; T_ghost_axis; T; T_ghost_wall;
42 T_ghost_wall2]; % (n_r+4, n_z)
43
44 T_ghost_in2 = -T(:, 2);
45 T_ghost_out2 = T(:, end);
46 T_zext2 = [T_ghost_in2, T_ghost_in, T, T_ghost_out, T_ghost_out2];
47 % (n_r, n_z+4)
48
49 % ---- radial convective flux (1/r) d(r*u*theta)/dr
50 -----
51 u_E = u(2:end, :);
52 u_I = u(1:end-1, :);
53
54 T_r_im2 = T_ext2(1:n_r, :);
55 T_r_im1 = T_ext2(2:n_r+1, :);
56 T_r_i = T_ext2(3:n_r+2, :); % = T
57 T_r_ip1 = T_ext2(4:n_r+3, :);
58 T_r_ip2 = T_ext2(5:n_r+4, :);
59
60 T_rf_o_upwind = upwind2_face(T_r_im1, T_r_i, T_r_ip1, T_r_ip2, u_E);
61 T_rf_i_upwind = upwind2_face(T_r_im2, T_r_im1, T_r_i, T_r_ip1, u_I);
62
63 rf_o = r_f(2:end);
64 rf_i = r_f(1:end-1);
65 rc = r_c;
66
67 conv_r = (rf_o .* u_E .* T_rf_o_upwind - rf_i .* u_I .*
68 T_rf_i_upwind) ./ (rc .* dr);
69
70 % ---- axial convective flux d(w*theta)/dz
71 -----
72 w_tp = w(:, 2:end);
73 w_bt = w(:, 1:end-1);
74
75 T_z_jm2 = T_zext2(:, 1:n_z);
76 T_z_jm1 = T_zext2(:, 2:n_z+1);
77 T_z_j = T_zext2(:, 3:n_z+2); % = T
78 T_z_jp1 = T_zext2(:, 4:n_z+3);
79 T_z_jp2 = T_zext2(:, 5:n_z+4);
80
81 T_zf_p_upwind = upwind2_face(T_z_jm1, T_z_j, T_z_jp1, T_z_jp2, w_tp)
82 ;
83 T_zf_m_upwind = upwind2_face(T_z_jm2, T_z_jm1, T_z_j, T_z_jp1, w_bt)
84 ;
85
86 % At the physical inlet face the incoming transported value is the
87 % prescribed Dirichlet value theta=0, not a LUD reconstruction. The
88 odd
89 % ghosts remain necessary for the second-order axial diffusion
90 stencil.
91 inflow = w_bt(:, 1) > 0.0;
92 T_zf_m_upwind(inflow, 1) = 0.0;
93

```

```

84     conv_z = (w_tp .* T_zf_p_upwind - w_bt .* T_zf_m_upwind) / dz;
85
86     % ---- radial diffusion (1/r) d/dr(r*dtheta/dr)
87     -----
88     visc_r = alpha * (rf_o .* (T_ext(3:end, :) - T) ...
89                     - rf_i .* (T - T_ext(1:end-2, :))) ./ (rc * dr^2);
90
91     % ---- axial diffusion d^2 theta/dz^2
92     -----
93     visc_z = alpha * (T_zext(:, 3:end) - 2.0 * T + T_zext(:, 1:end-2)) /
94     dz^2;
95
96     rhs = -conv_r - conv_z + visc_r + visc_z;
97 end

```

Listing 24: compute_rhs_T.m — Energy equation RHS: 2nd-order upwind convection, central diffusion

```

1 function T_new = advance_temperature(T, u, w, r_c, r_f, dr, dz, dt, Re,
2     Pr)
3 % ADVANCE_TEMPERATURE Advance temperature by one explicit Euler step.
4 % Thin wrapper around compute_rhs_T; the main time loop uses
5 % compute_rhs_T directly (via ab2_step) to build the AB2 update.
6 %
7 % Returns T_new : (n_r, n_z)
8
9 rhs = compute_rhs_T(T, u, w, r_c, r_f, dr, dz, Re, Pr);
10 T_new = T + dt * rhs;
11
12 % Inlet/outlet are enforced via ghost cells inside compute_rhs_T; no
13 % direct cell-value overwrite is needed.
14 end

```

Listing 25: advance_temperature.m — Temperature time advance

C.8 Convergence Monitoring

```

1 function res = compute_residuals(u_new, u_old, w_new, w_old, T_new,
2     T_old, r_c, r_f, dr, dz, dt)
3 % COMPUTE_RESIDUALS Compute convergence residuals.
4 %
5 % Returns res struct with fields:
6 %     .continuity : max |div(u)|
7 %     .momentum   : max change rate of both u and w (interior)
8 %     .temperature : max |T_new - T_old| / dt
9
10 div = compute_divergence(u_new, w_new, r_c, r_f, dr, dz);
11 res.continuity = max(abs(div(:)));
12 R_u = max(abs(u_new(2:end-1, :) - u_old(2:end-1, :)), [], 'all');
13 R_w = max(abs(w_new(:, 2:end-1) - w_old(:, 2:end-1)), [], 'all');
14 res.momentum   = max(R_u, R_w) / dt;
15 res.temperature = max(abs(T_new(:) - T_old(:))) / dt;
16 end

```

Listing 26: compute_residuals.m — Residual computation

```

1 function converged = check_convergence(res, tol_cont, tol_vel, tol_temp)
2 % CHECK_CONVERGENCE Return true when all residuals are below tolerances
3
4     converged = (res.continuity < tol_cont) && ...
5                 (res.momentum < tol_vel) && ...
6                 (res.temperature < tol_temp);
7 end

```

Listing 27: check_convergence.m — Convergence check

C.9 Analytical Reference Solutions

```

1 function w_an = analytical_velocity(r_star)
2 % ANALYTICAL_VELOCITY Hagen-Poiseuille fully developed velocity profile
3
4     %  $w^* = 2 * (1 - (2*r^*)^2)$ 
5
6     w_an = 2.0 * (1.0 - (2.0 * r_star).^2);
7 end

```

Listing 28: analytical_velocity.m — Hagen-Poiseuille axial velocity profile

```

1 function theta_an = analytical_temperature(r_star, z_star, Re, Pr)
2 % ANALYTICAL_TEMPERATURE Fully developed temperature profile (approved
3   form).
4
5   %  $\theta = 4*z^* + Re*Pr * [1/2*(2r^*)^2 - 1/8*(2r^*)^4 - 7/48]$ 
6   %
7   % Obtained by direct integration of the non-dimensional energy
8   % equation with
9   % the wall condition  $d\theta/dr^* = Re*Pr$ ; the constant  $-7/48$  follows
10  % from the
11  % exact global energy balance ( $\theta_{bulk} = 4*z^*$  for all  $z^* \geq 0$ ).
12  % Gives
13  %  $Nu = 48/11$ . Full derivation and sources: report, Appendix B.
14  % (An earlier transcription of the assignment omitted the  $1/2$  factor
15  % on the
16  % radial terms and used  $-3/4$ ; that form is not used anywhere in this
17  % codebase.)
18
19  eta = 2.0 * r_star;
20  theta_an = 4.0 * z_star + Re * Pr * (0.5 * eta.^2 - 0.125 * eta.^4 -
21    7.0/48.0);
22 end

```

Listing 29: analytical_temperature.m — Fully developed temperature profile ($Nu = 48/11$)

C.10 Plotting Helper

```

1 function cmap = diverging_bwr(n)
2 % DIVERGING_BWR Blue-white-red diverging colormap (no toolbox
3   dependency).
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

```

4 %   cmap = diverging_bwr(n) returns an n-by-3 colormap running from blue
5 %   (low) through white (mid) to red (high), suitable for signed fields
6 %   (e.g. radial velocity) plotted with a symmetric color axis centred
   on
7 %   zero via caxis([-lim, lim]).
8
9   if nargin < 1
10      n = 256;
11   end
12
13   blue  = [0.13, 0.40, 0.85];
14   white = [1.00, 1.00, 1.00];
15   red   = [0.80, 0.10, 0.10];
16
17   half = floor(n / 2);
18   lower = [linspace(blue(1), white(1), half)', ...
19           linspace(blue(2), white(2), half)', ...
20           linspace(blue(3), white(3), half)']';
21   upper = [linspace(white(1), red(1), n - half)', ...
22           linspace(white(2), red(2), n - half)', ...
23           linspace(white(3), red(3), n - half)']';
24   cmap = [lower; upper];
25 end

```

Listing 30: `diverging_bwr.m` — Blue–white–red diverging colormap for contour plots

References

- [1] R. K. Shah and A. L. London, *Laminar Flow Forced Convection in Ducts*, Supplement 1 to *Advances in Heat Transfer*, Academic Press, New York, 1978. (Circular duct, H1 boundary condition: $Nu = 48/11$.)
- [2] F. P. Incropera and D. P. DeWitt, *Fundamentals of Heat and Mass Transfer*, Wiley, New York. (Internal flow chapter: fully developed laminar flow in a circular tube with uniform surface heat flux, $Nu_D = 48/11 \approx 4.36$.)
- [3] W. M. Kays, M. E. Crawford, and B. Weigand, *Convective Heat and Mass Transfer*, McGraw-Hill, New York. (Fully developed laminar circular-tube flow, constant heat rate per unit length: $Nu = 48/11$.)
- [4] R. B. Bird, W. E. Stewart, and E. N. Lightfoot, *Transport Phenomena*, Wiley, New York. (The same fully developed constant-wall-flux tube problem is worked out and yields $Nu = 48/11$.)
- [5] H. Steiner, *Numerical Simulation and Modelling of Incompressible Flow*, lecture notes, WS 2025/26 (Sections I–V). Sec. III.3.1 *Direct linear equation solvers* (PDF pp. 55–61) presents (A) Gauss’ elimination (p. 55), (B) LU decomposition (pp. 56–57), (C) the Thomas algorithm (TDMA, p. 58), and (D) the Fourier method (pp. 59–61); Sec. III.3.2 (from p. 64) presents the iterative counterparts (Gauss–Seidel, SOR, LSOR, ADISOR). Included in this repository as `Assignemnt/NumMethods-WS2526-SecItoV.pdf`.